

基于 ECDSA 的匿名凭证

Matteo Frigo^{*} abhi shelat[†]

Google Google

matteof@google.com shelat@google.com

2024 年 12 月 21 日

摘要

匿名数字凭证允许用户证明其持有某个由身份颁发方所断言的属性，而无需透露关于自身的任何额外信息。例如，持有数字护照凭证的用户可以证明其“年龄 > 18 ”，而无需透露姓名或出生日期等其他属性。

尽管匿名凭证在隐私保护认证方面具有固有价值，但匿名凭证方案在大规模部署中一直面临挑战。困难的部分原因在于文献中的方案（如 BBS+ [10]）采用了新的密码学假设，要求对现有颁发方基础设施进行全系统变更。此外，颁发方通常要求数字身份凭证是设备绑定的，即将设备的安全元件纳入凭证展示流程。因此，BBS+ 等方案需要在每个用户设备上更新硬件安全元件和操作系统。

本文针对流行且已广泛部署的椭圆曲线数字签名算法（ECDSA）提出了一种新的匿名凭证方案。通过为 SHA256 相关陈述以及 ISO 标准化身份格式的文档解析添加高效的零知识（ZK）论证，我们的匿名凭证方案成为首个可以无需更改任何颁发方流程、无需更改移动设备、无需非标准密码学假设即可部署的方案。

生成关于 ECDSA 签名的 ZK 证明一直是其他 ZK 证明系统的瓶颈，因为 P256 等标准化曲线使用的有限域不支持高效的数论变换（NTT）。我们通过围绕求和校验和 Ligerio 论证系统设计 ZK 证明系统、设计在所需域上高效的 Reed-Solomon 编码方法，以及为 ECDSA 设计专用电路来克服这一瓶颈。

我们的 ECDSA 证明可在 60 毫秒内生成。当融合到 ISO MDOC 标准等完全标准化的身份协议中时，我们能够在移动设备上于 1.2 秒内（取决于凭证大小）生成 MDOC 展示流程的零知识证明。这些优势使我们的方案成为隐私保护数字身份应用的有力候选。

目录

1	引言	3
1.1	基于 ECDSA 的匿名凭证	4
2	零知识论证系统	6
2.1	可验证交互协议 (IP)	7
2.2	Ligero 零知识系统	9
2.3	ZK 协议	10
2.4	应用 Fiat-Shamir 变换	11
3	优化	12
3.1	素域上的 Reed-Solomon 编码	12
3.2	二元域上的 Reed-Solomon 编码	13
3.3	Ligero 证明大小的缩减	19
3.4	不同域上的电路输入一致性	19
4	电路设计	21
4.1	ECDSA 签名验证	21
4.2	SHA-256	22
4.3	CBOR 解析	24
5	评估	29
5.1	代数原语	30
5.2	SHA256 哈希验证	32
5.2.1	与 Ligero 的比较	32
5.2.2	与 Binius 的比较	34
5.2.3	与其他 ZK 系统的比较	35
5.3	ECDSA 签名验证	35
6	应用	36
6.1	简单匿名凭证	36
6.2	MDOC 匿名凭证	37
A	卷积插值	43
B	MDOC 标准	44

1 引言

David Chaum [12] 提出了匿名凭证系统（又称假名系统）的概念，作为赋予个人全面控制个人信息传播的一种方法。该系统由用户、颁发方和依赖方构成。颁发方可向用户颁发一组数字凭证；用户可向依赖方证明其持有满足某一特定属性的凭证，而无需透露超出“其所持凭证确实满足该属性”这一事实之外的任何信息。相互勾结的依赖方无法将特定用户与其参与的某对交互关联起来；即便是相互勾结的颁发方与依赖方，也不应能将特定用户与某次会话关联起来。

自 Chaum 提出这一概念后，早期基于特定签名方案的构造相继出现，但这些构造在使用上均存在限制。Lysyanskaya、Rivest、Sahai 和 Wolf [27] 设计了一种基于单向函数和通用零知识协议的方案，并提出了一种基于非标准离散对数假设的特定构造，该构造在凭证颁发与使用上同样有限制，无法实现不可链接性。Brands [8] 则基于证书引入了选择性披露系统。

Camenisch 和 Lysyanskaya [13] 提出了基于 RSA 的凭证系统，满足多次使用和不可链接性等匿名凭证所期望的属性，但其方案在一次基本展示中需要数百次模幂运算，并依赖于一个新的密码学假设。他们指出，匿名凭证方案的一个核心问题是设计一个同时具备高效知识证明的签名方案。后来，Camenisch、Drijvers 和 Lehmann 提出了 BBS+ 凭证方案 [10]，作为证明签名知识的高效方法，从而构建匿名凭证系统。该方案已被提议标准化，但它仍依赖于一个需要配对友好椭圆曲线的新密码学假设。

Kosba 等人 [22] 提出利用通用目的的零知识简洁非交互论证 (zkSNARKS) 基于标准化签名方案构建匿名凭证。在其方案之一中，凭证由一条 1KB 的消息构成，该消息由 RSA-PSS（采用 2048 位密钥和 SHA-256 哈希函数）签名。用户可在消息的某个值与外部值之间证明一个不等式（即基本的年龄大于 X 应用）。其证明者运行时间为 54–108 秒（取决于参数选择）。后来，Delignat-Lavaud、Fournet、Kohlweiss 和 Parno [15] 构建了一个 zkSNARK 系统用于证明 X509 证书（RSA 签名）的持有，该系统需要在 ZK 电路中解析 ASN.1 格式。其系统仅生成 RSA 签名证明需要 31 秒，整个匿名凭证展示则需要 61–160 秒。除了展示凭证的时间过长这一实际问题之外，这两个证明系统还需要昂贵的可信参数设置，并且都需要配对友好曲线（如 BBS+ 方案）。

沿着类似思路，Rosenberg、White、Garman 和 Miers [31] 利用应用于非标准化签名和哈希方案的通用 ZK 技术构建匿名凭证。其系统通过依赖区块链建立公共知识获得优势：颁发方每次向用户颁发凭证时必须扩展一个公共数据结构；用户在展示凭证时引用这一公共数据结构。由此，其系统将凭证展示问题归约为集合成员资格的 ZK 证明。此外，他们使用非标准哈希函数 (Poseidon) 以大幅提升集合成员资格 ZK 证明的效率。综合区块链的引入和 Poseidon 的使用，其证明系统的运行时间为 460–542 毫秒。采用（非标准）Schnorr 签名在（非标准）配对友好

曲线上的替代方案，其证明系统运行时间为 130 毫秒。其证明者最多需要 800MB 内存来生成证明，还需要一个由系统中所有颁发方和依赖方信任的、数百兆字节量级的可信参数，以及一个关于尚未标准化的双线性配对的计算困难性假设。

障碍 尽管已有若干将凭证系统部署于数字身份应用的尝试，这些努力在实践中遭遇了三个主要障碍。

第一个问题看似迂腐，实则影响深远。颁发凭证的机构已经采用并部署了经 NIST 等组织认证的标准化数字签名硬件和软件。具体而言，对于州级或国家级颁发方，许多颁发方已部署了仅支持 RSA 或 ECDSA 签名的硬件安全模块。通过非正式访谈可以发现，这些机构通常不愿或无力进行必要的基础设施投资，以支持更新的 CL 或 BBS+ 凭证系统，甚至无法支持配对友好曲线。即便是基于标准化密码学的当前匿名凭证方案也不够实用：如前所述，它们均需要数分钟的证明时间、大型可信设置参数，并且需要接受关于具有双线性配对的曲线的新密码学假设。

第二个问题是凭证通常由多个机构颁发（例如，每个州各自颁发驾驶证），而依赖方需要一个能在所有颁发方之间互操作的系统。通过一个涉及所有颁发方的仪式生成单一公共可信参数的任务看起来极为困难且难以实现。迄今为止，所有基于标准化密码学的解决方案都需要可信参数，因此在部署上颇具挑战。

第三个，也是最关键的障碍是匿名凭证可能被大规模滥用。如果凭证可以从一个恶意用户复制给另一个用户，那么验证凭证的系统可能无法提供任何有效的健全性保证。与传统凭证不同——传统凭证一旦被发现复制或伪造可以被识别并吊销——匿名凭证不会在展示协议中泄露任何信息（除了共享属性之外）。因此，仅凭观察展示流程来撤销此类凭证更加困难或不可能实现。即使在仅提供选择性披露属性而不具备完全匿名性的证书系统中，这一问题也会出现。

为解决这种凭证复制问题，数字身份凭证的颁发方通常要求用户将其凭证保存在某种形式的可信硬件中。现代移动设备通常包含一个安全元件，这是一种据称具有防篡改、抗侧信道攻击等特性的专用硬件模块，能够抵御大多数密钥提取攻击。例如，ISO MDOC 标准中的身份协议规定了一种颁发和展示流程，在接收设备上验证安全元件的存在，并要求安全元件参与每次展示流程。不幸的是，这些安全元件在现场难以更换，且目前仅支持 RSA 和 ECDSA 等传统标准化签名方案。特别是，它们不支持在配对友好曲线、格密码方案上执行操作，甚至不支持 EDDSA 等已得到广泛部署的签名方案。

1.1 基于 ECDSA 的匿名凭证

为了解决上述所有不足，即（a）效率、（b）使用标准化签名方案和哈希函数的要求，以及（c）无需公共知识或可信设置，我们提出了一种构建于 ECDSA 签名之上的高效匿名凭证方案。

具体而言，遵循先前工作的方法，我们首先构建一个高效的 ECDSA 签名知

识证明，然后开发颁发方向用户颁发一组凭证或属性所需的其他工具。接着，我们将这些工具组合成一个协议，使得用户可以向依赖方展示其持有这些凭证中任意子集的所有权。

与先前工作不同，我们提出的 ECDSA 签名知识证明在运行时间上与 BBS+ 证明相当，且仅依赖于标准化 SHA256 哈希函数的抗碰撞安全性。我们方法的关键在于结合两种近期 ZK 证明技术：一种基于求和校验的优化可验证计算协议，以及基于交互式预言机证明 (IOP) 模型（更精确地说是交互式 PCP，即 IPCP 模型）的 Ligerio ZK 证明系统。在近期 ZK 构造中使用求和校验的优势已得到证明：它无需对给定电路的完整见证值执行昂贵的数论变换 (NTT)，而是可以在定理验证电路大小的线性时间内运行基于求和校验的验证协议。此外，基于求和校验的可验证计算协议在信息论意义上是安全的，因为它们无需任何计算假设。类似地，IPCP 模型中的 Ligerio 证明系统需要实例化一个安全的证明预言机，这可以通过仅依赖密码学哈希函数的 Merkle 树来实现。在实践中，我们以非交互方式实例化我们的协议，并依赖随机预言机模型来获得健全性和零知识属性。

关键成果 证明 ECDSA 签名知识的一个关键瓶颈与已标准化的椭圆曲线有关，具体而言与这些曲线所在有限域的性质有关。用于 ECDSA 的标准化曲线（如 P256）的设计未考虑 NTT 的需求，即这些曲线所在的有限域不具备高效 NTT 算法所需的单位根。因此，先前的工作本质上是在另一个为 ZK 证明优化的有限域中模拟 P256 域的算术运算。即使使用非常巧妙的算术技巧，这一开销也高达 40–100 倍的算术运算增量。

我们通过多种方式克服这一瓶颈。首先，我们为 P256 等素数的二次域扩展设计了专用 NTT，使得 NTT 的计算代价大约是在同等规模 NTT 友好素域上执行的 4 倍。¹ 请注意，这一额外代价使得最小化见证值的大小变得更加重要。

我们还开发了一种新颖的 Reed-Solomon 编码方法，该方法允许利用域的子域结构，与标准 NTT 方法相比在证明大小上实现常数倍的改进。具体而言，大多数项目将 Reed-Solomon 码定义为多项式在单位根处的赋值，而我们将码定义为多项式在基域中算术序列点处的赋值。我们提供了一种通过线性卷积在这些点上执行插值的方法。因此，当我们的见证值属于某个小的子域或基域时，即便 NTT 本身在更大的扩展域上执行，最终码字仍属于该子域，从而得到更小的证明。

在某些情况下，我们需要纳入关于某些布尔密集函数（如 SHA256）原像的零知识论证。受 Diamond 和 Posen [16, 17] 工作的启发，我们认识到利用扩展域 $GF(2^k)$ 执行求和校验可带来固有的加速优势。在这种情况下，标准 NTT 算法不再适用，我们转而使用加法 FFT 族 [24]。具体而言，我们开发了一种新的双向截断加法 FFT，用于解决陪集上多项式插值的问题。如我们的基准测试所示，该操作在这些基本代数运算上实现了近一个数量级的加速。

¹我们相信这一思路也可被其他证明系统所采用。

我们结果的下一个组成部分来自于我们为验证 ECDSA 签名、验证 SHA-256 原像以及解析 CBOR 和 JSON 等嵌套文档格式（这些格式常见于当前身份凭证标准如 ISO MDOC 中）所开发的专用电路。

我们最后一项贡献是一种在凭证验证任务同时涉及 ECDSA 签名和 SHA-256 哈希时实现最佳性能的方法。所有先前的 ZK 证明系统在单一有限域 $\mathbb{F}$ 上验证算术电路。然而，如我们的基准测试所示，在二元域上验证 SHA-256 原像的性质比在素域上快近一个数量级。

关于本应用的特别说明 一些近期 ZK NARK 系统能够在数百字节的量级上产生非常小的证明，但这些系统需要可信参数设置仪式。如上所述，在涉及多个颁发方、依赖方和用户的匿名凭证系统中，我们认为这样的设置仪式过于难以组织，因此我们专注于不需要任何可信设置的 ZK 解决方案。

此外，尽管我们开发的系统在证明大小上是简洁的，但许多近期 ZK 系统在验证者时间上也是简洁的。然而，在匿名凭证应用中，保证证明者时间更为重要，因为证明者通常是用户的移动设备。因此，我们系统的目标是减少证明者时间和能耗。

2 零知识论证系统

我们通过将 Ligerio 证明系统与基于求和校验协议的公币可验证计算（VC）协议相组合，构建一个零知识论证（ZKARG）系统。VC 协议允许证明者在时间和空间均小于 $|C|$ 的条件下，向验证者证明 $C(x) = 0$ 。值得注意的是，自求和校验引入以来 [26]，这类协议一直隐含于文献之中。

我们构建 ZK 论证的方法遵循 Hyrax [40] 的相同模式，后者本身是 Ben-Or 等人 [7] 思想的实例化。具体而言，在电路 C 、公共输入 x 和见证值 w 上，证明者首先向 w 及承诺参数发送承诺 com 。证明者和验证者随后针对输入 (C, x) 的 VC 协议交互地产生一个承诺抄本 T ；具体而言，当 VC 协议指示证明者向验证者发送第 i 条消息 m_i 时，它转而向该消息发送承诺 t'_i ；验证者始终以随机挑战响应证明者第 i 个承诺，遵循常规公币 VC 协议，并推迟执行所有规定的验证步骤。最后，证明者和验证者针对定理“(t'_1, ..., t'_\ell) 是对抄本 t 的承诺，且 t 是 (C, x) 的一个接受 VC 抄本”执行零知识证明。特别地，该定理的 ZK 证明隐含地确保了验证者的所有 VC 步骤对于原始抄本 t 均成立。

这一组合享有效率优势。由于 VC 对 C 产生简洁证明，我们组合系统中的 Ligerio 证明比直接对 $C(x) = 0$ 的独立 Ligerio 证明更小。如下所示，这显著提升了整体系统性能，因为 Ligerio 中最昂贵的组件 NTT，被应用于 $C(x) = 0$ 计算的小型抄本，而非 $C(x)$ 的完整表格。请注意，VC 协议不是零知识协议，但 Ligerio 是，因此在这种组合下产生的协议满足零知识属性。

在本节的其余部分，我们详细解释各个组件，并给出完整协议及其性能的形式化描述。

2.1 可验证交互协议 (IP)

我们描述一种可验证计算协议，它源自围绕求和校验协议的 VC 研究工作的一系列成果 [18, 14, 33, 39, 40]。

求和校验 求和校验交互协议 [26] 允许证明者高效地向验证者证明，某个声称的值等于一个多项式在布尔超立方体每个点处的求值之和。这里我们引用 Thaler [34] 中的一个性能定理来刻画求和校验。

定理 2.1 ([34, 命题 4.1]). 设 g 是定义在有限域 $\mathbb{F}$ 上的 v 元多项式，每个变量次数至多为 d 。对于任意指定的 $H \in \mathbb{F}$ ，设 $\mathcal{L}_H$ 为满足如下条件的多项式 g （作为预言机给出）的语言：

$$H = \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \cdots \sum_{b_v \in \{0,1\}} g(b_1, \dots, b_v). \quad (1)$$

求和校验协议是 $\mathcal{L}_H$ 的交互证明系统，完备性误差 $\delta_c = 0$ ，健全性误差 $\delta_s \leq vd/|\mathbb{F}|$ 。

进而，若 $\deg(g) = O(1)$ ，则协议需要 v 轮通信，总通信代价为 $O(v)$ 个域元素，验证者时间为 $O(v) + T(g)$ ，证明者时间为 $O(M(g))$ ，其中 T 表示对 g 进行 1 次赋值计算的代价， M 表示在超立方体上对 g 求值的代价。

作为符号上的便利，我们经常用大写字母表示 $\{0,1\}$ 上的变量向量。例如，令 $B = (b_1, \dots, b_v)$ ，我们写作：

$$H = \sum_{B \in \{0,1\}^v} g(B)$$

或者甚至以 $H = \sum_B g(B)$ 作为公式 (1) 的简写。

将求和校验应用于电路验证 Thaler [34] 对求和校验协议应用于可验证计算任务的历史和实践进行了全面综述。特别地，如 GKR [18] 所首先展示的，求和校验可应用于验证 $C(x) = 0$ 的任务，其中 C 是一个以第 0 层为输出层、第 d 层为输入层的分层电路。

我们提出这一类协议的一种优化变体，建立在 Cormode 等人 [14]、Thaler [33] 以及后续工作 [39, 40] 的观察之上。我们的改进包括：简化连线谓词、不同的逐层归约方法，以及用于计算稀疏连线谓词赋值的不同方法。

在我们的表述中，电路的第 j 层根据输入导线数组 $V_{j+1}[0, \dots, w_{j+1} - 1]$ （即第 $j+1$ 层的输出）计算输出导线数组 $V_j[0, \dots, w_j - 1]$ 。与 GKR [18] 中分离的加法门和乘法门不同，我们只有一种“门”，用于计算通用的四次型 (quad)：

$$V_j[i] = \sum_{\ell, r} Q_j[i, \ell, r] \cdot V_{j+1}[\ell] \cdot V_{j+1}[r],$$

并针对这种电路结构专门化求和校验。我们称 Q 为四次型 (quad), $Q_j[i, \ell, r]$ 为四次项 (quad term)。

按照惯例, 所有层的输入导线 $V_j[0] = 1$, 因此四次型处理了统一方式下的经典加法门和乘法门。例如, 这种四次型方法将标准椭圆曲线加法公式的深度从 5 降至 3。按照惯例, 若所有输出层的输出 $V_0[i]$ 均为 0, 则输入满足电路。

上述数组自然地指定了多变量——特别是多线性——多项式。我们使用波浪线符号来表示数组的唯一多线性扩展。例如, $\tilde{V}_j(i)$ 表示对数组 $V_j[i]$ 进行插值的唯一多线性扩展。

设 $\omega_j = \lceil \log_2(w_j) \rceil$ 。在协议开始时, 验证者选取随机点 $r_0 \in \mathbb{F}^{\omega_0}$ 。由于所有输出期望为 0, 验证者期望 $\tilde{V}_0(r_0) = 0$ 。我们的过程从两个断言 $(c_{0,0}, c_{0,1}) = (0, 0)$ 开始, 关于 $\tilde{V}_0(r_0)$ 的值。在第一步, 两个断言均关于同一赋值点的值, 但在后续轮次中, 这些断言将关于不同点的值。

协议 2.2. $\text{Verify}_{P,V}(C, x)$, 用于定理 $C(x) = 0$

深度为 d 的电路 C 定义在域 $\mathbb{F}$ 上。证明者和验证者持有输入 (C, x) 。设 $(Q_0, \dots, Q_{d-1})$ 为 C 每层的四次型, w_j 为第 j 层的导线数, $\omega_j = \lceil \log_2(w_j) \rceil$ 。设 $V_j[i]$ 表示第 j 层第 i 条输出导线的值。验证者若任何检查失败则拒绝。

1. 验证者采样 $r \leftarrow \mathbb{F}^{\omega_0}$, 设 $r_{0,b} \leftarrow r$, $c_{0,b} \leftarrow 0$, $b \in \{0, 1\}$ 。
2. 对 $j = 0, 1, \dots, d-1$: // 证明者断言 $c_{j,b} = \tilde{V}_j(r_{j,b})$, $b \in \{0, 1\}$ 。

(a) 验证者发送 $\alpha_j \leftarrow \mathbb{F}$, 设 $Y = \alpha_j \cdot c_{j,0} + (1 - \alpha_j)c_{j,1}$ 。

(b) 定义 $2\omega_j$ 元多项式

$$g_j(L, R) = [\alpha_j \cdot \tilde{Q}_j(r_{j,0}, L, R) + (1 - \alpha_j) \tilde{Q}_j(r_{j,1}, L, R)] \cdot \tilde{V}_{j+1}(L) \cdot \tilde{V}_{j+1}(R)$$

其中 $L = L_0, \dots, L_{\omega_j-1}$, $R = R_0, \dots, R_{\omega_j-1}$ 。

(c) 证明者和验证者对以下断言应用求和校验:

$$Y = \sum_{L, R \in \{0,1\}^{\omega_j}} g_j(L, R)$$

求和校验协议产生两个随机点 ℓ 和 r , 一个值 y , 以及断言 $y \stackrel{?}{=} g_j(\ell, r)$ 。

(d) 证明者发送断言 $c_{j+1,0} = \tilde{V}_{j+1}(\ell)$ 和 $c_{j+1,1} = \tilde{V}_{j+1}(r)$ 。

(e) 验证者检查:

$$y \stackrel{?}{=} [\alpha \cdot \tilde{Q}_j(r_{j,0}, \ell, r) + (1 - \alpha) \tilde{Q}_j(r_{j,1}, \ell, r)] \cdot c_{j+1,0} \cdot c_{j+1,1}$$

并以点 $(r_{j+1,0}, r_{j+1,1}) = (\ell, r)$ 和断言 $c_{j+1,b}$ 继续。

3. 验证者通过直接计算 $\tilde{V}_d(r_{d,b})$ 来验证断言 $c_{d,b} \stackrel{?}{=} \tilde{V}_d(r_{d,b})$ 。

定理 2.2. 设 C 是一个深度为 d 、定义在 $\mathbb{F}$ 上的分层电路，第 j 层宽度为 w_j ，设 $\omega_j = \lceil \log(w_j) \rceil$ ， $Z = 2 \sum_{i=1}^d \omega_i$ 。

$\text{Verify}_{P,V}(C, x)$ 是定理 $C(x) = 0$ 的 Z 轮交互协议，完备性误差为 0，健全性误差为 $O(Z/|\mathbb{F}|)$ ，总通信量为 $\Theta(Z)$ 个域元素。

此外，验证者计算包括：从 $\mathbb{F}$ 采样 $\Theta(Z)$ 个元素，执行 $\Theta(Z)$ 次域运算，以及在输入 x 和由随机挑战导出的张量形式的向量之间执行一次点积。

证明者计算由 $O(|C|)$ 次域运算构成。

注记 实现所述运行时间需要对证明者和验证者的求和校验进行细致但已知的优化，以及一种高效计算每层连线谓词 $\tilde{Q}$ 赋值的方法。

为了方便，记 $\text{sz}(C, x)$ 为该协议在实例 (C, x) 上的抄本长度。

可以清楚地看出，步骤 2e 可以移出循环。此外，验证者在步骤 2(c) 中于求和校验执行的所有检查也可以移到循环之外。在这一表述中，验证者可以分为两部分：包含步骤 1、步骤 2(a) 和步骤 2(c) 中随机采样步骤的交互部分，以及执行步骤 2(c)、2e 和步骤 3 中所有检查的第二部分。为方便起见，我们用 $\text{check}_V(C, x)$ 表示第二部分。

SIMD 优化 众所周知，求和校验可用于利用连线谓词中的正则性。例如，当 C 验证同一较小电路的多个实例或副本时，可以实现验证者简洁性 [14, 33, 39]。这些扩展可以通过在我们的表述中引入一个 *copy* 变量来加入，但由于我们的系统不需要验证者简洁性，故省略。

2.2 Ligerio 零知识系统

Ligerio [2] 是一种零知识交互式 PCP (ZKIPCP)，它是“多方头脑中计算” (MPC-in-the-head) 技术的一种优化实例化。证明大小大约是电路大小的平方根，且证明系统仅假设密码学哈希函数的存在。注意，IPCP 是 IOP 的一种特例，其中证明者仅在第一轮发送一个证明预言机，在后续轮次中证明者简单地响应验证者的挑战消息（而非预言机）。²

与此前我们考虑的仅含输入 x 的电路 C 不同，从这一点开始，我们将输入进一步分拆为一对 (x, w) ，其中 x 是证明者和验证者均已知公共输入， w 是仅证明者知道的私密见证值。

我们的 ZK 协议使用 Ligerio 来包裹上一节中的 check_V 验证者，即它可被解释为高效可验证计算协议与通用零知识协议的组合。

为方便起见，我们识别 Ligerio 协议的若干组成部分。LigerioCommit(x, w) 方法允许证明者对输入 (x, w) 产生一个短承诺 com。接下来，证明者和验证者分别

²将证明者后续通信视为消息而非预言机（仅包含单点），使得从 IOP 到协议的编译更加简单和高效。

运行 $\text{LigeroProve}(C, x, w, \text{com})$ 和 $\text{LigeroVerify}(C, x, \text{com})$, 最终验证者方法输出接受或拒绝。

Ames 等人 [2] 刻画了 Ligero 的性能如下:

定理 2.3 ([2, 推论 5.3]). 固定参数 n, m, l, k, t, e 使得 $e < (n - k)/2$ 且 $n > 2k + e$ 。设 $C: \mathbb{F}^n \rightarrow \mathbb{F}$ 为大小为 s 的算术电路, 其中 $|\mathbb{F}| \geq l + n$, $m \cdot l > nl + s$ 且 $k > l + t$ 。则协议 $\text{Ligero}(C, \mathbb{F})$ 是一个具有如下性质的非交互证明:

- **自适应知识误差** $\varepsilon_{\text{FSK}}(C, T, \lambda) = T \cdot \varepsilon(C) + 3(T^2 + 1)/2^\lambda$, 对抗 T 次查询证明者;
- **自适应知识误差** $\Omega(T \cdot \varepsilon_{\text{FSK}})$, 对抗 $T - O(q(C) \log(|C|))$ 次查询量子证明者;
- **统计零知识** $z'(C, \lambda) = 4|C|/2^{\lambda/4}$;
- **效率**: V 进行 t 次大小为 $\mathbb{F}^{4m+5\sigma}$ 的符号查询。 P 向 V 通信的域元素数目为 $\sigma \cdot n + \sigma(k + l - 1) + \sigma(2k - 1)$ 。

其中

$$\varepsilon(C) = \varepsilon_K(C) = \max\left(\frac{n+2}{|\mathbb{F}|^\sigma}, (1-e/n)^t\right)$$

$$|C| = n + (k + l - 1) + (2 \cdot k - 1)$$

2.3 ZK 协议

我们现在描述我们的新 ZK 协议。

协议 2.5. ZK 协议 (P, V) , 用于电路 $C(x, w) = 0$

电路 C 定义在域 $\mathbb{F}$ 上。证明者持有输入 $(C, (x, w))$, 验证者持有输入 (C, x) 。

1. P, V 计算大小 $\ell \leftarrow \text{sz}(C)$ (即 Verify 抄本的长度)。
2. P 计算增强见证值 $w' \leftarrow (w, \text{pad})$, 其中 $\text{pad} \leftarrow \mathbb{F}^\ell$ 。
3. P 计算 $\text{com} \leftarrow \text{LigeroCommit}(w')$ 并将 com 发送给 V 。
4. P, V 开始运行 $\text{Verify}_{P,V}(C, (x, w))$, 进行如下修改:
 - 每当 Verify_P 指示证明者向验证者发送第 i 条消息 $m_i \in \mathbb{F}$ 时, P 转而发送 $t'_i \leftarrow m_i + \text{pad}_i$ 。
 - 当 Verify_V 指示验证者发送第 i 个随机挑战 r_i 时, V 返回 r_i 。

最终, P, V 对实例 $(C, (x, w))$ 持有一个加密的求和校验抄本 $T' = ((t'_i, r_i))_i$ 。

5. 定义电路 C' , 先解密抄本 $T = ((t_i = t'_i - \text{pad}_i, c_i))_{i \in [0, \ell]}$, 然后运行 $\text{check}_V(C, (x, w), T)$ 。
6. P, V 分别运行协议方法 $\text{LigeroProve}(C', (x, w'), T', \text{com})$ 和 $\text{LigeroVerify}(C', x, T', \text{com})$; V 输出结果。

注记

- 第 2 步中证明者仅承诺 $|(x, w)| + \ell$ 个元素。这一大小通常远小于 $|C|$, 而若将 Ligero 直接应用于 C , 证明者必须承诺 $|C|$ 个元素。这是一个重要的性能优势, 因为 Ligero 承诺涉及 NTT, 这是证明系统中唯一的非线性时间组件。当 $\mathbb{F}$ 不具备合适的单位根时 (如 ECDSA 电路的情况), 这一改进尤为重要。
- LigeroProve 被应用于电路 C' , 该电路解密抄本 (通过一次性填充, 即每个 T' 元素一次域运算), 然后运行 check_V 电路。这个检查电路执行少量域运算, 然后计算 (x, w) 与由验证者发送的随机挑战导出的公开可计算挑战向量之间的一次点积。(这一公共向量也具有张量形式, 由 Verify 协议诱导。) 总体而言, Ligero 证明系统被用于验证一个关于原始电路 C 的小型陈述。

定理 2.4. 若 C 是定义在 $\mathbb{F}$ 上的深度为 d 、宽度为 w 的电路, 且 Ligero 是具有自适应知识健全性误差 $\varepsilon_{\text{FSK}}(|C|, \lambda)$ 和统计零知识 $z'(C, \lambda)$ 的零知识协议, 则协议 2.5 是诚实验证者零知识协议, 完备性和知识健全性误差为 $O(|C|, |\mathbb{F}|) + d \cdot \log(w)/|\mathbb{F}|$, 零知识性 $z'(C, \lambda)$ 。

证明. (梗概) 提取误差由 Ligero 提取器失败和验证协议信息论健全性间隙的联合界给出。零知识属性直接由在增强电路 C' 上运行的 Ligero 模拟器给出。 $\square$

2.4 应用 Fiat-Shamir 变换

为从基本协议构造非交互协议, 我们应用 Fiat-Shamir 变换, 并通过 Canetti 等人 [9] 提出的逐轮健全性引理论证健全性。

定理 2.5 ([9]). 设 $\Pi = (P, V)$ 是语言 L 上的 $2r$ 消息公开交互证明, 具有完美完备性、 $\text{poly} \log(n)$ 比特的证明者-验证者通信总量, 以及具有对应状态函数 State 的逐轮健全性。设 X_n 表示部分抄本的集合 (包括输入和已发送的所有消息), Y_n 表示当 Π 在长度为 n 的输入上执行时验证者消息的集合。若哈希族 $\mathcal{H} = \{H_n : X_n \rightarrow Y_n\}$ 是 R_{State} -相关不可提取的且可在时间 $\tilde{O}(n)$ 内计算, 则存在 $(\text{Gen}, \tilde{P}, \tilde{V})$ 构成 L 的自适应可靠公开可验证论证。

Canetti 等人 [9] 给出了为什么求和校验是逐轮健全的, 参数为 $O(d/|\mathbb{F}|)$ 的论证。Ligero 也是逐轮健全的, 参数为 $O(\eta(C')/|\mathbb{F}|)$ 。我们的组合也是逐轮健全的, 参数类似。

3 优化

3.1 素域上的 Reed-Solomon 编码

Ligero 多项式承诺专门需要 Reed-Solomon 码，而非一般的纠错码。Ligero 将消息（一个域元素数组）解释为某个低次多项式在特定点处的赋值，并依赖于码字由同一多项式在其他某些点处的赋值构成这一事实。评估点的选择对健全性无影响，但会影响性能。本节描述我们对素域 $\mathbb{F}_q$ 所做的选择，并在第 3.2 节中讨论二元域。

一组自然的评估点是 ω^i (ω 为单位根)，从而可以使用基于 FFT 的方法进行插值。然而，这一选择会导致 $(q-1)$ 的因子分解、码长以及插值效率之间复杂的相互依赖关系。特别地，对于 $\mathbb{F}_{p256}$ (P256 是 NIST 素数 $P256 = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$)，该域没有可用的单位根。

为避免这一复杂性，我们做出如下选择。一个消息由次数至多为 d 的多项式 $p(x)$ 在 $d+1$ 个点 $x \in \{\mathcal{F}_q(i) : i \leq 0 \leq d\}$ 处的赋值构成，其中 $\mathcal{F}_q(i) = i \bmod q$ 是将整数 i 自然注入域 $\mathbb{F}_q$ 的映射。类似地，长度为 m 的码字由 m 次赋值 $\{p(\mathcal{F}_q(i)) : 0 \leq i < m\}$ 构成。这些定义对特征足够大的域是良定义的，唯一剩余的问题是计算多项式插值。

如 [14, 第 2.1 节] 所述，我们的总体策略是将多项式插值归约为卷积，然后应用最方便的卷积算法。具体而言，我们使用如下恒等式（对整数成立）：对于整数 $k > d$,

$$p(k) = (-1)^d \cdot (k-d) \binom{k}{d} \cdot \sum_{0 \leq i \leq d} \left(\frac{1}{k-i} \right) \cdot (-1)^i \binom{d}{i} p(i). \quad (2)$$

参见附录 A 对公式 (2) 的证明。³ 除了常数缩放之外，公式 (2) 确实是（线性）卷积：将输入 $p(i)$ 逐元素乘以常数 $(-1)^i \binom{d}{i}$ ，与核 $(k-i)^{-1}$ 进行卷积，再逐元素除以常数 $(-1)^d \cdot (k-d) \binom{k}{d}$ ，即得 $p(k)$ 。公式 (2) 在 $k < q$ 时也成立（模 q ）。

整数上的卷积可以用 $O(n \log n \log \log n)$ 次域运算通过 Nussbaumer [29] 算法计算（亦见 [21, 习题 4.6.4-59]）。若域是 FFT 友好的（即其乘法群的阶因子分解为小素数），则卷积可以用 $O(n \log n)$ 次域运算通过某种形式的 FFT 计算。

在我们的情形中，ECDSA 签名标准化于少量椭圆曲线，如 P256、P384、P521。每条曲线定义于特定的基有限域 $\mathbb{F}_q$ (q 为素数)，这些域对 FFT 不友好。然而，每个奇素数 q 都具有 $q = r \cdot 2^\ell - 1$ 的形式，而二次扩展 $\mathbb{F}_{q^2}$ 包含 2^ℓ 阶单位根，因为乘法群的阶为 $q^2 - 1 = (q+1)(q-1)$ ，且 2^ℓ 整除 $(q+1) = r \cdot 2^\ell$ 。虽然这一事实对平凡情形 $\ell = 0$ 无用，但对于 ECDSA 素数，可以选取足够大的 ℓ ，使得在扩展域上的 FFT 可行。Zhang 等人 [42] 在 q 为 Mersenne 素数 $q = 2^\ell - 1$ 的情形下做了同样的观察。

³除了常数缩放之外，公式 (2) 实际上是矩阵分解 $V_0^{-1} V_1 = D_0 C D_1$ 的显式特殊情形，其中 V_i 是 Vandermonde 矩阵， C 是 Cauchy 矩阵， D_i 是对角矩阵 [20]，定理 1。

因此, 至少对于 ECDSA 素数, 我们通过二次扩展中的两次 FFT 在基域中计算卷积, 假设预先计算了卷积核的变换。尽管对于这种情形存在专用算法 (大致相当于实数输入的复数 FFT 算法, 或有限域类比的离散 Hartley 变换), 我们简单地在扩展域中完整地执行卷积, 与更专用的算法相比产生约两倍的额外开销。

3.2 二元域上的 Reed-Solomon 编码

概述 在特征为 2 的域中, $1 + 1 = 0$, 第 3.1 节中的插值点选择不再适用。我们转而遵循 [24, 17] 的方式, 采用下文描述的不同选择。

$\text{GF}(2^k)$ 构成 $\text{GF}(2)$ 上的 k 维向量空间。一次性固定这个向量空间的一组基 β_i , $0 \leq i < k$ 。(关于基的具体选择, 见第 3.3 节。) 对 j 的二进制表示中的第 i 位写作 j_i , 即 $j = \sum_{0 \leq i < k} j_i 2^i$, $j_i \in \{0, 1\}$, 我们将整数 j 注入域元素 $\mathcal{F}(j)$, 将 j 的各位解释为基 β_i 下的坐标:

$$\mathcal{F}(j) = \sum_{0 \leq i < k} j_i \beta_i. \quad (3)$$

然后, 与第 3.1 节中的方式类似, 我们假设待编码的消息由次数至多为 d 的多项式 $p(x)$ 在 $d+1$ 个点 $x \in \{\mathcal{F}(i) : 0 \leq i \leq d\}$ 处的赋值构成。类似地, 长度为 m 的码字由 m 次赋值 $\{p(\mathcal{F}(i)) : 0 \leq i < m\}$ 构成。

注入 $\mathcal{F}(j)$ 的特定选择 (相对于例如 $\mathcal{F}(j) = \omega^j$) 的动机是加法 FFT 的存在, 它在 $O(m \log d)$ 次运算中解决了插值问题。

我们使用 Lin、Chung 和 Han [24] 首先提出的加法 FFT, 该算法建立于先前渐近复杂度更高的算法之上。Lin 等人定义了一种新多项式基, 作为常用单项基 x^i 的替代, 并给出了在子空间中所有 d 个点处计算次数至多为 $(d-1)$ 的多项式赋值的算法 ($d = 2^\ell$), 以及用新基表示的多项式。该算法可以反向执行, 以从赋值重构新基中的系数。参见 [23, 25] 对同一思路的替代介绍。

Diamond 和 Posen [17] (算法 2) 将该算法重新表述为三重嵌套循环, 类似于 Cooley-Tukey FFT 的常见实现。⁴ 此外, Diamond 和 Posen 将算法推广, 以获得次数小于 2^ℓ 的多项式在 $2^{\ell+\mathcal{R}}$ 个点处的赋值, 而 [24] 在子空间的陪集上对 2^ℓ 个点赋值。

出于我们的目的, 我们希望避免 d 和 m 必须是 2 的幂次这一限制, 即便加法 FFT 算法仅对 2^ℓ 大小的输入有效。对于次数为任意 d 的多项式赋值, 我们使用 Diamond-Posen 的三重嵌套循环表述 ($\mathcal{R} = 0$), 并附加 [24] 中的原始陪集参数。我们将系数数组用零填充至大小 2^ℓ , 并通过求足够多陪集处的赋值来获得所需的 m 次赋值, 可能舍弃不需要的点。

在给定 $d+1$ 个赋值的情形下, 重构新基的多项式系数更具挑战性, 当 $d+1$ 不是 2 的幂次时。我们不能简单地将赋值数组补 $2^\ell - (d+1)$ 个零然后应用逆加法

⁴Cooley-Tukey FFT 需要对输入或输出进行位反转, 但 Diamond-Posen 算法不需要, 因此两种算法并不完全相同。

FFT, 因为这会产生次数 $2^\ell - 1$ (而非 d) 的多项式。为解决这一问题, 我们将 van der Hoeven [35] 的截断 FFT 适配为加法 FFT 算法。截断 FFT 计算 FFT 输出的子集, van der Hoeven [35] 给出了一个优雅的逆过程算法。尽管截断 FFT 文献主要关注节省域运算或避免存储隐含零元素, 我们更倾向于将该算法视为双向 FFT, 如下所示: 将线性变换视为

$$\begin{bmatrix} Y_0 \\ Y_1 \end{bmatrix} = A \begin{bmatrix} X_0 \\ X_1 \end{bmatrix},$$

其中 X_i 和 Y_i 是 n_i 个元素的列向量。对于 A 是 Fourier 矩阵或其加法变体的具体情形, 正向 FFT 在给定 X 的条件下计算 Y , 逆 FFT 在给定 Y 的条件下计算 X 。我们称一个给定 Y_0 和 X_1 计算 Y_1 和 X_0 的算法为双向 FFT。因此, 双向 FFT 算法可以计算正向或逆变换, 以及两者的多种组合。

加法 FFT 的细节 遵循 Diamond 和 Posen [17] 引入的符号, 设 U_i 为基的前缀 $\{\beta_k : 0 \leq k < i\}$ 的张成空间。定义子空间消没多项式 $W_i(x) = \prod_{u \in U_i} (x - u)$ 及其归一化变体 $\tilde{W}_i(x) = W_i(x)/W_i(\beta_i)$ 。

多项式 W_i 是线性化的: $W_i(x + y) = W_i(x) + W_i(y)$ 。参见 [37, 引理 2.3] (该引理不加证明地给出) 和 Mateer 论文 [28, 定理 15] (有证明)。线性化对于加法 FFT 的效率至关重要, 并且允许高效计算 $W_i(x)$: 若 $x = \sum_k x_k \beta_k$ 用基表示, 则 $W_i(x) = \sum_k x_k W_i(\beta_k)$ 。

此外, 我们有 $W_{i+1}(x) = W_i(x)W_i(x + \beta_i)$, 因为 $U_{i+1} = U_i \cup (\beta_i + U_i)$, 因此 $W_{i+1}(x) = W_i(x)(W_i(x) + W_i(\beta_i))$ 由线性化得到。结合基本情形 $W_0(x) = x$, 这两个性质允许高效计算所有必要的 $W_i(\beta_j)$ 和 $\tilde{W}_i(\beta_j)$ 。我们预计算并存储 $\tilde{W}_i(\beta_j)$ 于数组 $\tilde{W}[i][j]$ 中。

加法 FFT [24] 对所有 x 是某个陪集 $U_i + \alpha$ 的赋值, 计算多项式 $p(x) = \sum_{0 \leq k < 2^i} c_k X_k(x)$, 给定多项式以新多项式基 $X_k(x)$ 展开的系数 c_k 。反向运行该算法 (逆 FFT) 可从赋值计算系数。Lin 等人 [24, 方程 6] 给出了基 $X_k(x)$ 的显式公式, 但基的精确形式对插值无关紧要, 只要 $X_k(x)$ 是 k 次多项式即可。也就是说, 可以从 U_i 上的赋值出发, 计算系数, 然后在陪集 $U_i + \alpha$ 上赋值以得到新的赋值, 而无需知道基的精确形式。⁵

算法 1 给出我们的加法 FFT 实现的伪代码, 大体上来源于 [17] (算法 2), 并包括其逆。我们展示过程 TWIDDLE, 它计算文献中所谓的“旋转因子”, 但在实际实现中我们通过递归式 $\text{TWIDDLE}(i, u + 2^k) = \tilde{W}[i][k] + \text{TWIDDLE}(i, u)$ 同时计算所有旋转因子, 这给出了一种将旋转因子数组大小加倍的线性时间算法。

⁵这种情形类似于 Newton 有限差分插值方法, 其中从赋值出发计算一批中间值, 并在额外点处计算多项式赋值, 从未提及 Newton 基, 但某种意义上可将该方法解释为用 Newton 基表示多项式的展开。

Algorithm 1: 加法 FFT, 改编自 [17] (算法 2)

```

1  procedure FFT( $\ell, \alpha, B[\ ]$ )
    要求: 大小为  $2^\ell$  的数组  $B$ , 包含多项式展开
         $p(x) = \sum_{0 \leq k < 2^\ell} B[k]X_k(x)$  (以新多项式基  $X_k(x)$  表示) 的系数。
    确保:  $B[k] \leftarrow p(\mathcal{F}(k + \alpha))$ , 其中  $\mathcal{F}(\cdot)$  由公式 (3) 定义。
2  for 所有  $i: 0 \leq i < \ell$ , 按  $i$  降序执行 do
3       $s \leftarrow 2^i$ 
4      for 所有  $u: 0 \leq 2s \cdot u < 2^\ell$  do
5           $t \leftarrow \text{TWIDDLE}(i, 2s \cdot u + \alpha)$ 
6          for 所有  $v: 0 \leq v < s$  do
7              BUTTERFLY-FWD( $B, 2s \cdot u + v, s, t$ )
8          end
9      end
10 end
11 end

12 procedure IFFT( $\ell, \alpha, B[\ ]$ )
    输入: 过程 FFT 的逆。
13 for 所有  $i: 0 \leq i < \ell$ , 按  $i$  升序执行 do
14      $s \leftarrow 2^i$ 
15     for 所有  $u: 0 \leq 2s \cdot u < 2^\ell$  do
16          $t \leftarrow \text{TWIDDLE}(i, 2s \cdot u + \alpha)$ 
17         for 所有  $v: 0 \leq v < s$  do
18             BUTTERFLY-BWD( $B, 2s \cdot u + v, s, t$ )
19         end
20     end
21 end
22 end

23 procedure TWIDDLE( $i, u$ )
24     设  $u_k$  为  $u$  的第  $k$  位, 即  $u = \sum_k u_k 2^k$ 。
25     return  $\sum_k u_k \cdot \tilde{W}[i][k]$ 
26 end

27 procedure BUTTERFLY-FWD( $B[\ ], w, s, t$ )
28      $B[w] \leftarrow B[w] + t \cdot B[w + s]$ 
29      $B[w + s] \leftarrow B[w + s] + B[w]$ 
30 end

31 procedure BUTTERFLY-BWD( $B[\ ], w, s, t$ )
32      $B[w + s] \leftarrow B[w + s] - B[w]$ 
33      $B[w] \leftarrow B[w] - t \cdot B[w + s]$ 
34 end

```

双向 FFT 的细节 忽略位反转置换，加法 FFT 的计算图与普通 Cooley-Tukey FFT 的计算图相同（见图 1）。给定新多项式基中的 n 个输入系数 $C[\cdot]$ ，一个基本蝶形运算的循环将输入分解为偶数和奇数数组 $C_e[\cdot]$ 和 $C_o[\cdot]$ ，对其递归地应用变换，最终得到赋值数组 $E[\cdot]$ 。

在双向 FFT 设置中，输入由以下部分构成： E 的前缀 $E[i]$ ($0 \leq i < k$) 的赋值，以及 C 的后缀 $C[i]$ ($k \leq i < n$) 的系数。目标是计算剩余的系数和赋值。为此，我们将 van der Hoeven [36]（第 6 节）的讨论适配至加法 FFT 的情形。

考虑过程 BUTTERFLY-FWD 中描述的初等正向蝶形操作。设 $(a, b) = (B[w], B[w+s])$ 是应用蝶形操作前的值， $(c, d) = (B[w], B[w+s])$ 是应用操作后的值。则蝶形运算计算

$$\begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} 1 & t \\ 1 & t+1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}. \quad (4)$$

类似地，过程 BUTTERFLY-BWD 中的反向蝶形操作计算逆变换

$$\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} t+1 & -t \\ -1 & 1 \end{bmatrix} \begin{bmatrix} c \\ d \end{bmatrix}. \quad (5)$$

van der Hoeven [36]（方程 7）关于双向 FFT 算法的关键思路是对角蝶形（过程 BUTTERFLY-DIAG），它在 b 上正向执行而在 c 上反向执行：

$$\begin{bmatrix} a \\ d \end{bmatrix} = \begin{bmatrix} -t & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} b \\ c \end{bmatrix}. \quad (6)$$

由 (a, d) 计算 (b, c) 的对称情形也是可能的，并且对于普通 Cooley-Tukey FFT 是可行的 [36, 方程 8]。然而，加法 FFT 对应的方程

$$\begin{bmatrix} b \\ c \end{bmatrix} = (1+t)^{-1} \cdot \begin{bmatrix} -1 & 1 \\ 1 & t \end{bmatrix} \begin{bmatrix} a \\ d \end{bmatrix} \quad (7)$$

要求 $(1+t)$ 可逆，而在我们的情形中这不总是成立。这就是为什么我们选择赋值的前缀（而非系数的前缀）已知这一约定的原因。

考虑图 1，假设 $k \geq n/2$ 。因此，顶部 $N/2$ -FFT 块的整个输出是已知的，整个 $C_e[\cdot]$ 可以从 $E[\cdot]$ 的已知前缀的逆 FFT 中获得。给定 $C_e[\cdot]$ 和已知的后缀 $C[i]$ ($i \geq k - n/2$)，对角蝶形计算 $C_o[i]$ ($i \geq k - n/2$)。结果是一个大小为 $n/2$ 的双向 FFT 问题： C_o 的长度 $n/2 - (k - n/2)$ 的后缀以及 E 底部一半的长度 $k - n/2$ 的前缀。这一双向子问题递归地求解，从而计算缺失的 E 元素和 C_o 。给定整个 C_e 和 C_o ，缺失的 C 通过反向蝶形计算。

对 $k < n/2$ 的情形类似的考虑也适用。完整的双向 FFT 算法的伪代码见算法 2。

即使 FFT 算法将大小为 n 的问题归约为两个大小为 $n/2$ 的子问题，双向 FFT 算法也会产生一个大小为 $n/2$ 的双向 FFT 问题和另一个大小为 $n/2$ 的问题

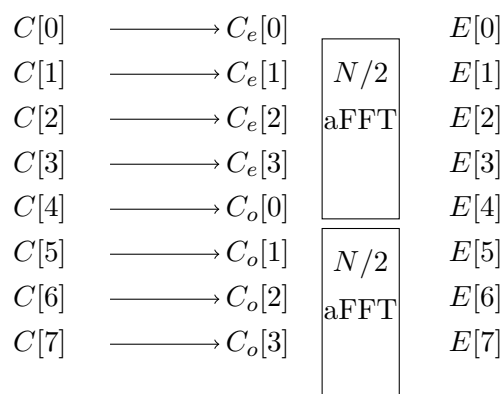


图 1: 普通加法 FFT 的流程图, 忽略位反转置换时与 Cooley-Tukey FFT 的流程图相同。具有两个输入和两个输出的”蝶形”表示输出是输入的某个 2×2 线性变换。Lin 等人 [24] 绘制了一个略有不同的图, 将线性变换分解为三角矩阵, 但对于我们的目的, 这一层次的细节并不重要。

(全正向或全逆向 FFT)。因此, 假设后者变换有优化实现, 递归的开销是最小的: $O(\log n)$ 次递归调用和 $O(\log n)$ 次旋转因子计算。在实践中, 双向 FFT 的性能本

质上与完全正向或完全逆向 FFT 相同。⁶

Algorithm 2: 双向 FFT, 改编自 [36] (第 6 节)

```

1 procedure BIDIRECTIONAL-FFT( $i, \alpha, k, B[]$ )
    要求: 长度为  $n = 2^i$  的数组  $B$ , 其前  $k$  个元素是某多项式的赋值, 其
        余元素是同一多项式 (以新多项式基表示) 的系数。
    确保: 将  $B$  覆写为一个数组, 其前  $k$  个元素为系数, 其余元素为赋值。
2   if  $i > 0$  then
3        $i \leftarrow i - 1$ 
4        $s \leftarrow 2^i$ 
5        $t \leftarrow \text{TWIDDLE}(i, \alpha)$ 
6       if  $k < s$  then
7           for 所有  $v: 0 \leq v < s$  do
8               | BUTTERFLY-FWD( $B, v, s, t$ )
9           end
10          BIDIRECTIONAL-FFT( $i, \alpha, k, B$ )
11          for 所有  $v: 0 \leq v < k$  do
12              | BUTTERFLY-DIAG( $B, v, s, t$ )
13          end
14          FFT( $i, \alpha + s, B[s:]$ )
15      end
16      else
17          IFFT( $i, \alpha, B$ )
18          for 所有  $v: k - s \leq v < s$  do
19              | BUTTERFLY-DIAG( $B, v, s, t$ )
20          end
21          BIDIRECTIONAL-FFT( $i, \alpha + s, k - s, B[s:]$ )
22          for 所有  $v: 0 \leq v < k - s$  do
23              | BUTTERFLY-BWD( $B, v, s, t$ )
24          end
25      end
26  end
27 end

```

Reed-Solomon 编码的细节 我们的原始插值问题: 输入次数为 d 的多项式 $p(\mathcal{F}(i))$ 在 $0 \leq i \leq d$ 处的 $d+1$ 个赋值, 必须计算 $p(\mathcal{F}(i))$ 在 $0 \leq i < m$ (d 和 m 任意, 不必须是 2 的幂次) 处的值。设 ℓ 为满足 $d < 2^\ell$ 的最小整数。运行双向 FFT 算法,

⁶稍加注意, 递归可以完全消除, 但我们发现无递归算法不必要地令人困惑。

$n = 2^\ell$, $k = d + 1$, 假设前 k 个系数未知, 其余为零。双向 FFT 给出前 $n - k$ 个缺失赋值和全部 k 个系数。然后对尽可能多的陪集运行正向 FFT, 以产生 m 个赋值。

3.3 Ligerio 证明大小的缩减

健全性要求使用大域, 如 $\text{GF}(2^{128})$ 。我们将 $\text{GF}(2^{128})$ 实现为 $\text{GF}(2)[x]/(Q(x))$, 其中 $Q(x) = x^{128} + x^7 + x^2 + x + 1$ 。使用这种 $Q(x)$ 的选择, x 是域乘法群的生成元。

我们系统中几乎所有消息都由在 $\{0, 1\}$ 中的见证位构成, 这些位被打包进大小小于 2^{14} 的数组中。回忆第 3.2 节中的 Reed-Solomon 编码: 若消息是 $\text{GF}(2^{128})$ 某个子域中的元素数组, 且评估点也是同一子域中的元素, 则整个码字由子域元素构成, 可以紧凑表示。为确保所有评估点都在子域中, 选取 β_i 为子域的基即可。

因此, 我们选择 $\text{GF}(2^{16})$ 作为 $\text{GF}(2^{128})$ 的子域, 令 $g = x^{(2^{128}-1)/(2^{16}-1)}$ 为 x 对 $\text{GF}(2^{16})$ 的生成元, $\beta_i = g^i$ ($0 \leq i < 16$) 为子域的基。这使得我们在承诺方案中可以用 16 位代替 128 位来编码元素。当我们在选择时, 我们通过公式 (3) 将见证位和小整数注入子域。

由于我们不显式地用域表示子域, 我们面临确定一般域元素 $u \in \text{GF}(2)[x]/(Q(x))$ 是否属于子域的问题, 以及若属于, 确定其用基 β_i 表示的展开 $u = \sum_{0 \leq i < 16} u_i \beta_i$ 的问题。

我们通过标准 Gauss 消元法解决这一问题。我们将 $\text{GF}(2^{128})$ 的元素视为 $\text{GF}(2)$ 上 128 个元素的行向量, 对应于单项基 x^j , 这一约定允许通过单个 128 位异或指令将两个行向量相加。然后, 我们构造矩阵 B , 其第 i 行由行向量 β_i 构成。利用这些约定, 我们需要对 x 求解矩形方程组 $xB = u$, 这可以通过对 B 进行预备 LU 分解 (将 B 化为行阶梯形) 再标准解三角方程组完成。当 $n = 16$ 时, 解的代价为 $O(n)$ 次行运算, 这是高效的, 因为行运算仅需单个宽异或指令。在我们的系统中, 投影到子域所花费的时间可以忽略不计。

在当前实现中, 子域优化仅在 I/O 期间序列化和反序列化域元素时减少证明大小。我们在完整域中执行所有算术运算。Diamond 和 Posen [17] 主张使用一种使子域显式化的塔式表示, 并可以在可能时利用子域算术。我们将两种策略的比较留作未来工作。

3.4 不同域上的电路输入一致性

逐轮健全性属性要求求和校验和验证协议使用大小至少为 2^{128} 的域。Ligerio 承诺要求大小至少为 $O(\sqrt{|(x, w)|})$ 的子域, 其中 (x, w) 是电路输入的大小。因此, 通常最高效的做法是在足够大的基域的域扩展上运行求和校验。然而, ECDSA 电路施加了不同的约束: 高效验证 P256 曲线上的椭圆曲线运算要求电路定义在特定

的 256 位素域上。

考虑定理陈述 (e, pk) , 它验证存在消息 m 和签名值 (r, s) , 使得 $e = \text{SHA256}(m)$, 且 (r, s) 是 m 在公钥 pk 下的签名。在 $\mathbb{F}_{p^{256}}$ 上的单一电路会产生在比健全性所需更大的素域上验证 m 的 SHA256 哈希的开销。类似地, 在 $\mathbb{F}_{2^{128}}$ 上的电路会产生约 $100\times$ 的开销来模拟椭圆曲线的算术 (相对于原生域)。

一种更高效的自然方法是在最优的 $\mathbb{F}_{p^{256}}$ 域中执行 ECDSA 验证实例 (e, pk) , 并在最优的二元域中执行 SHA256 验证实例 (e, m) 。其中的问题是, 在两个不同电路中, 一个作弊的证明者可能对 e 使用 2 个不一致的值。在本节中, 我们描述一种确保在两个或多个证明域、或更一般地两个或多个证明系统中使用同一定理时, 输入一致性的技术。

我们确保输入一致性的解决方案是: 用一个关于公共输入 e 的消息认证码 (MAC) 验证来增强两个域中的每个电路。我们使用信息论 MAC $\text{MAC}_{a,b}(x) = ax + b$, 其中运算在 $\mathbb{F}_{2^{128}}$ 上进行。注意, 在 ECDSA 电路中, 这要求在素域 $\mathbb{F}_{p^{256}}$ 上模拟 $\mathbb{F}_{2^{128}}$ 的算术。对于小输入, 这一计算几乎没有额外开销。

关于如何选择密钥 k 是一个次要问题; 允许验证者选择 k 可能破坏零知识性; 允许证明者选择 k 会破坏健全性, 因为证明者可以故意选择一个允许两个不同 e 值具有相同 MAC 值的密钥。在我们的方案中, 该密钥由证明者和验证者联合采样。具体地, 证明者首先承诺其份额 $k_p = (a_p, b_p)$ 以及秘密输入 e 。验证者接下来随机采样 k_v 并明文发送给证明者。证明者然后计算密钥 $k = k_p + k_v$, 用 k 计算共享输入 e 的 MAC, 并将 MAC 值 m_e 发送给验证者。验证者将 m_e 用作两个电路 C_h 和 C_{sig} 的公共输入。两个电路都包含验证 $m_e = \text{MAC}(e)$ 的检查。

协议 3.1. 双域 ZK 协议 (P_2, V_2)

设电路 $C_1(x_1, w_1, w)$ 定义在域 $\mathbb{F}_1$ 上, $C_2(x_2, w_2, w)$ 定义在 $\mathbb{F}_2$ 上, $w \in \{0, 1\}^c$ 为两者的公共输入, 以自然方式嵌入各自的 $\mathbb{F}_i$ 中, $x_1, w_1 \in \mathbb{F}_1^*$, $x_2, w_2 \in \mathbb{F}_2^*$ 。双方持有 (x_1, x_2) , 证明者另外持有 (w_1, w_2, w) 。

- 定义电路 $\hat{C}_i((x_i, m, k_v), (w_i, w, k_p))$, 其中 $k_p, k_v \in \{0, 1\}^{256}$, 如下:
 - 断言 $C_i(x_i, w_i, w) = 0$;
 - 断言 $\text{MAC}_k(w) = m$, 其中 $k = k_p \oplus k_v$, 所有计算在 $\text{GF}(2^{128})$ 上进行。
- P, V 对电路 $\hat{C}_1, \hat{C}_2$ 并行运行协议 2.5, 但对步骤 3 进行如下修改:

3. P 采样 $k_p \leftarrow \text{GF}(2^{128})^2$, 然后计算

$$\text{com}_i \leftarrow \text{LigeroCommit}((w_i, w, k_p, \text{pad}_i), \mathbb{F}_i)$$

并发送给 V 。

- (a) V 采样 $k_v \leftarrow \text{GF}(2^{128})^2$ 并发送给 P 。
 (b) P 计算 $m \leftarrow \text{MAC}_{k_p \oplus k_v}(w)$ 并发送给 V 。

定理 3.1. 若 C_1 和 C_2 是定义在域 $\mathbb{F}_1$ 和 $\mathbb{F}_2$ 上的深度为 d 、宽度为 w 的电路，且 MAC_k 是具有统计安全性 λ_s 的信息论消息认证码，若协议 2.5 是具有自适应知识健全性误差 $\eta(|C_1|, |C_2|)$ 的零知识协议，则协议 3.1 是具有完备性和知识健全性误差 $O(\eta(|C_1|, |C_2|) + \lambda_s)$ 的零知识协议。

证明. (梗概) 修改后的协议本质上是协议 2.5 的并行组合。为构造这一组合协议的知识健全性提取器算法，对 C_1 和 C_2 分别运行提取器。若任一提取器失败，则组合提取器失败。假设第一个提取器输出 $(x_1, w_1, w, \text{pad}_1)$ ，第二个提取器输出 $(x_2, w_2, w', \text{pad}_2)$ 。若 $w \neq w_1$ ，则组合提取器失败。否则，输出 (x_1, x_2, w_1, w_2) 并停止。

通过对各个提取器失败概率以及 MAC 的统计健全性参数的标准联合界论证，组合提取器以压倒性概率成功。此外，组合提取器的运行时间本质上是各个提取器运行时间之和，因此在恶意证明者预言机的运行时间上是多项式时间的。

零知识性方面，模拟器随机采样 m 并对 C_1, C_2 运行分段模拟器。 $\square$

4 电路设计

4.1 ECDSA 签名验证

本节描述一个算术电路的构造，该电路验证哈希消息 $e = H(m)$ 在公钥 (pk_x, pk_y) 下的 ECDSA 签名的存在性。遵循 Sec-1 [11] 规范（与 ANSI X9.62.2005 标准相同），ECDSA 签名验证过程如下：

Algorithm 3: ECDSA-Verify($Q, H(m), r, s$) [11]

- 1 1: 从 $H(m)$ 中导出整数 e 。;
 - 2 2: 验证 $r, s \in [1, q - 1]$ 。;
 - 3 3: 计算 $u_1 = es^{-1} \bmod q$ 和 $u_2 = rs^{-1} \bmod q$ 。;
 - 4 4: 计算 $R = (r_x, r_y) = G \cdot u_1 + Q \cdot u_2$ 并验证 $R \neq \text{id}$ 。;
 - 5 5: 验证 $r = (r_x \bmod q)$ 。;
-

上述若干步骤在整数上或在域 $\mathbb{F}_q$ 中进行，而我们的检查必须在域 $\mathbb{F}_p$ 中进行。为此，算法 4 中定义的验证电路在输入 x 上成功，当且仅当某个容易计算的元组 $t(x)$ 能使算法 3 成功执行。此外，该电路几乎是完美完备的，仅有少数情形不能通过我们的检查但能通过官方规范——这在我们考虑的所有应用（包括存在恶意签名者的情形）中均是可接受的。

Algorithm 4: 验证电路 $V(Q, H(m), r, s, r_y)$

-
- 1 1: 从 $H(m)$ 中导出整数 e 。;
 - 2 2: 验证 $r, s \in [1, q - 1]$ 。;
 - 3 3: 验证 $Q, R = (r, r_y) \in E$ 且 $R \neq \text{id}$ 。;
 - 4 4: 验证 $\text{id} = G \cdot e + Q \cdot r - R \cdot s$ 。;
-

两个过程的主要区别在于，后者通过在群中执行多标量幂运算并进行额外检查以确保点在椭圆曲线群上，从而避免了模 q 的逆元计算。

定理 4.1. 若 $V(Q, H(m), r, s, r_y)$ 成功，其中 $r, s \in [0, 2^{\lceil \log p \rceil} - 1]$, $r_y \in \mathbb{F}_p$ ，则 $\text{ECDSA}(Q, H(m), r, s)$ 成功。

验证多标量乘法 算法 4 中最昂贵的步骤是步骤 4，即验证 $G \cdot e + Q \cdot r - R \cdot s$ 等于单位元。通过提供中间值作为见证值，我们产生一个利用民间 Shamir 技巧的简单扩展计算左端的低深度电路。具体而言，计算由一个标准的“加倍并相加”循环构成，循环遍历指数的各位。在第 i 次迭代中，位 e_i, r_i, s_i 被用来索引一个包含 G, Q, R 元素的 8 个组合的表。当前累加器被加倍，并加上表查找的结果。如前所述，第 $i+1$ 次迭代的输入依赖于第 i 次迭代的输出，导致深度较大的电路。因此，我们将每次迭代的输出作为见证值提供，并将基本加倍并相加块的输出与下一块的输入进行比较。为减小深度，7 个非零表项以见证值的形式给出，并由电路验证。ECDSA 验证的结果电路大小的完整计算见表 1。

表 1: ECDSA 验证的电路大小与深度。电路编译器利用共享项，因此总计略小于各部分之和。

	深度	四次型数	项数	输入数
多标量乘法	7	19,534	37,550	1,038
范围检查 + 其余	12	5,475	10,569	1,038
合计	12	23,453	47,598	1,038

4.2 SHA-256

SHA-256 哈希函数由一系列压缩函数迭代构成，压缩函数由 Ch 、 Maj 、 Σ_0 和 Σ_1 函数以及最后的模 2^{32} 加法组成。一条 n 位消息 m 首先被分割为 64 字节的块；基本块函数被应用于第 i 块以产生压缩后的 32 字节输出，然后与第 $i+1$ 块组合。最后一个消息块最多为 55 字节，填充零后附加一个 8 字节的 n 编码。

为验证至多 $64 * k - 9$ 字节的消息，我们的电路必须评估 k 个 SHA-256 基本块函数。一个基本块函数以 8 个 32 位状态值（用字母 $A-H$ 表示）、消息的 64 字

节作为输入，并产生一个新的状态值。该块使用 Ch 、 Maj 、 Σ_0 和 Σ_1 函数：

$$\begin{aligned} Ch(E, F, G) &= (E \wedge F) \oplus (E \wedge \neg G) \\ Maj(A, B, C) &= (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C) \\ \Sigma_0(A) &= (A \ggg 2) \oplus (A \ggg 13) \oplus (A \ggg 22) \\ \Sigma_1(E) &= (E \ggg 6) \oplus (E \ggg 11) \oplus (E \ggg 25) \end{aligned}$$

状态值 B, C, D, F, G, H 分别被赋予 A, B, C, E, F 的值。值 A, E 更新为：

$$\begin{aligned} A &= W_i \text{ 田 } K_i \text{ 田 } H \text{ 田 } Ch(E, F, G) \text{ 田 } \Sigma_1(E) \text{ 田 } Maj(A, B, C) \text{ 田 } \Sigma_0(A) \\ E &= W_i \text{ 田 } K_i \text{ 田 } H \text{ 田 } Ch(E, F, G) \text{ 田 } \Sigma_1(E) \text{ 田 } Maj(A, B, C) \text{ 田 } D \end{aligned}$$

其中 田 表示模 2^{32} 加法。

验证模加法 模 2^{32} 加法的验证可以通过某种布尔加法器电路来实现，无论是串行进位还是并行前缀形式。事实上，若 A 、 B 和 C 全部作为输入提供，则加法 $A \text{ 田 } B = C$ 的验证可以由常数深度电路完成，方法是利用以下事实：第 i 位的进位可以从方程 $C[i] = A[i] \oplus B[i] \oplus \text{carry}[i]$ 中推断，因此只需验证进位已被正确传播，而不必实际传播进位。

然而，至少在某些域中，利用原生域算术执行加法会产生更小的电路。我们在电路中使用这种方法的两个变体，它们都利用了可以用至多 t 个项 $T[i]$ 之和模 2^{32} （其中具体地 $t \leq 7$ ）计算的事实。

若域的特征大于 $2^{32} \cdot t$ ，则 t 个项在域算术中的加法不会溢出。设 $s = \sum_{0 \leq i < t} T[i]$ 为域算术中的和， r 为声称的模 2^{32} 意义下的和，解释为域元素。设 $z = s - r$ 在域中。则 $z = 0 \pmod{2^{32}}$ 当且仅当 $\prod_{0 \leq i < 2^{32}t} (z - 2^{32}i) = 0$ ，而后者的检查通过域中乘法树来实现。⁷

前一验证方法在特征较小的域（如 $\text{GF}(2^{128})$ ）中不起作用。在这种情形中，我们将前一方法应用于域的乘法群。设 g 为乘法群的生成元，假设 $\text{order}(g) > 2^{32}t$ 。则 $g^s = \prod_{0 \leq i < t} g^{T[i]}$ ， t 个可能的值 $g^{r+2^{32}i}$ ($0 \leq i < t$) 之一，可通过域中的乘法树来比较（如前）。

表 2 报告了我们在不同域中 SHA256 电路实例化的大小，并使用不同的打包参数。为减少输入数量，从而减小承诺大小，我们可以将多个位证明值打包进单个见证值中，然后应用简单电路从见证值中提取各个位。

⁷在域具有（乘法） 2^{32} 阶单位根的特殊情形中，我们可以选取 g 为这样的单位根，从而“免费”获得模 2^{32} 的算术。这一特殊情形不适用于我们电路使用的域。

表 2: 1 个 SHA-256 块在两种不同域上的电路大小和深度。打包列指示每个见证值打包（以及在电路中提取）的位数。

	打包	深度	四次型数	项数	输入数
$\mathbb{F}_{P_{256}}$	—	7	37,974	167,348	6,657
	2	9	65,690	215,504	3,585
	3	10	76,287	239,919	2,625
	4	11	85,732	273,556	2,049
$\mathbb{F}_{2^{128}}$	—	13	53,435	87,642	6,657
	2	14	65,727	150,991	3,585
	3	15	73,818	166,494	2,625
	4	16	79,607	177,959	2,049

4.3 CBOR 解析

MDOC 凭证以简洁二进制对象表示 (CBOR) [3] 格式编码。出于本文的目的，⁸ 一个 CBOR 数据元要么是整数（有符号或无符号）、布尔值、字节数组、CBOR 数据元的数组，要么是从数据元到数据元的映射。

CBOR 旨在同时最小化编码文档的大小和解析器的大小。为此，数据元编码的第一个字节由三位标签（表示各种类型：整数、数组、映射等）和五位计数构成。小整数直接编码在计数中。对于较大的整数，计数编码整数的字节大小，随后在文档的后续字节中存储该整数。若标签表明数据元是数组或映射，计数则编码数组中的元素数量或映射中的键值对数量，其实际内容存储在后续字节中。

设 $in[i]$ 为第 i 个输入字节，其中 $in[i]$ 本身是一个包含 8 条输入线的数组，每条线携带 1 位。在我们的证明系统的上下文中，“解析 CBOR”并不意味着将 $in[]$ 转换为抽象语法树，而是解析电路预期断言如下形式陈述的真实性：

顶层文档是一个映射。顶层映射包含一个键为字符串 `valueDigests`、值为另一个映射的键值对。`valueDigests` 映射包含一个键为字符串 `org.iso.18013.5.1`、值为另一个映射的键值对。`org.iso.18013.5.1` 映射包含一个键为无符号整数 4、值为长度 32 的字节数组（即某个 SHA256 哈希）的键值对……

为设计电路以做出这类断言，我们的策略是将相关数据元在输入中的位置作为见证值提供给电路。虽然叶节点比较（如“位置 i 处的键是字符串 `valueDigests`”）相对直接，但结构断言（如“位置 i 处的键值对是从位置 j' 开始的映射的条目”）要求电路重构文档的树结构。在本节的其余部分，我们重点介绍计算这类重构的技术。

⁸CBOR 还支持若干其他情形，如浮点数，我们忽略这些情形。

在“组合”算术电路模型中，整个输入被提供给固定大小的电路，因此文档大小被限制为某个在电路生成时已知的常数 N 字节。类似地，由于模型不包含支持任意嵌套数组和映射的栈，我们将解析器限制为 L 个嵌套层次（ L 为某个常数）。

玩具 CBOR 为说明 CBOR 解析中的主要问题和解决方案，我们将讨论集中于解析如下定义的一种无上下文 CBOR 类玩具语言。

一个玩具 CBOR 文档由一系列字节 $in[i]$ 构成，这些字节被分组为词元。从位置 i 开始的词元定义如下：

情形 $0x00 \leq in[i] < 0x10$ ：词元为 $\text{Int}(in[i])$ ，即四位整数。下一词元从位置 $i + 1$ 开始。

情形 $in[i] = 0x10$ ：词元为 $\text{Int}(in[i + 1])$ ，即存储在 $in[i + 1]$ 中的八位整数。下一词元从位置 $i + 2$ 开始。

情形 $0x20 \leq in[i] < 0x30$ ：词元为 $\text{Array}(in[i])$ 。下一词元从位置 $i + 1$ 开始。

情形 $in[i] = 0x30$ ：词元为 $\text{Array}(in[i + 1])$ 。下一词元从位置 $i + 2$ 开始。

否则：错误。

除了词元的词法结构之外，玩具 CBOR 还具有上下文无关的句法结构，我们通过给出一个递归下降解析器来非正式地定义它。具体地，解析器从流中读取第一个词元并消费它。若词元为 $\text{Int}(k)$ ，解析器返回整数 k 。否则词元为 $\text{Array}(k)$ ，此时解析器递归地调用自身 k 次，返回包含递归调用结果的长度为 k 的数组。

完整的 CBOR 既具有更丰富的词元结构（包括可变长度整数和字符串），也具有更丰富的句法结构（包括数组和映射），但主要困难在我们的玩具版本中已经出现。

词法分析器 第一步是识别词元边界。给定长度为 N 的输入 in ，算法 5 产生一个数组 $header$ ，使得当且仅当词元从位置 i 开始时 $header[i]$ 为真。这样，“数组从位置 i 开始”的断言必须先断言 $header[i]$ 。

我们在算法 5 中省略了过程 DECODE，因为它复杂但不具有启发意义。可以将其视为组合逻辑，对每个输入位置 i ，将 $in[i]$ 的位解码为一组信号（表明词元是整数还是数组），并计算域值 $length[i]$ ，表示从位置 i 开始的词元的长度，使下一词元从位置 $i + length[i]$ 开始。对于玩具 CBOR， $length[i] \in \{1, 2\}$ 可以仅由 $in[i]$ 计算，而真实 CBOR 需要查看 $in[i + 1]$ 乃至后续字节。由于是电路而非过程，DECODE 预测性地对所有位置 i 计算 $length[i]$ ，与位置 i 是否真正有词元开始无关。在实际实现中，DECODE 还必须处理无效词元并输出有效性状态，并且我们必须断言 $header[i]$ 蕴含位置 i 处词元的有效性。

Algorithm 5: 分词的序列算法

```

1 procedure TOKENIZE( $N, in[]$ )
2    $(token, length) \leftarrow \text{DECODE}(N, in)$  // 过程 DECODE (未显示) 产生数
   组  $length$ , 其中  $length[i]$  是从  $in[i]$  开始的词元的字节长度。
3    $header[0] \leftarrow \text{true}$ 
4    $slen[0] \leftarrow length[0]$ 
5   for  $i = 1$  to  $N - 1$  do
6      $header[i] \leftarrow ((slen[i - 1] - 1) = 0)$ 
7     if  $header[i]$  then
8        $slen[i] \leftarrow length[i]$ 
9     end
10    else
11       $slen[i] \leftarrow (slen[i - 1] - 1)$ 
12    end
13  end
14  return  $(token, header)$ 
15 end

```

减少词法分析深度 在电路生成时, 若 N 的上界已知, 则可以展开算法 5 中的循环并生成电路。然而, 这样的电路深度为 $O(N)$, 对于基于求和校验的证明者而言效率较低。我们现在利用熟知的并行前缀技术来减少电路深度。

若 $header[i]$ 始终为假 (在算法 5 的第 9 行, 这不是实际情形), 则 $slen$ 的更新方程为 $slen[i] = slen[i - 1] + A[i]$ ($A[i] = -1$), 这是前缀和或扫描的一个实例。由于 $slen[i] = \sum_{j \leq i} A[j]$, 每个 $slen[i]$ 可以通过为每个 i 构建独立的加法树简单地以深度 $O(\log n)$ 计算。各种统称为并行前缀的算法允许同时以深度 $O(\log n)$ 计算所有 i 的 $slen[i]$, 同时最大化公共子表达式的复用。详见 Blelloch [6] 及其中的参考文献。

完整的更新方程 (公式 (4), 此时 $A[i] = -1$) 为分段扫描的形式, 即:

$$slen[i] = \text{if } header[i] \text{ then } length[i] \text{ else } slen[i - 1] + A[i]. \quad (4)$$

存在对数深度电路可以计算给定 $header[\cdot]$ 、 $length[\cdot]$ 和 $A[\cdot] = -1$ 的 $slen[\cdot]$ 。参见 [6, 第 1.5 节] 将分段扫描变换为无分段扫描的通用技术。

分段扫描技术可以解决电路深度问题, 但分段扫描要求段标记 $header[\cdot]$ 作为电路的输入提供, 而在我们的情形中, $header[i]$ 是 $slen[i - 1]$ 的函数 (在算法中的第 7 行计算)。为解决这个循环依赖, 我们将布尔值 $header[i]$ 作为见证值提供, 每个输入字节位置一个, 并使用算法 6 作为起点。

Algorithm 6: 带见证值的分词算法

```

1 procedure TOKENIZEW( $N, in[ ], header[ ]$ )
2    $(token, length) \leftarrow \text{DECODE}(N, in)$ 
3    $slen[0] \leftarrow length[0]$ 
4   for  $i = 1$  to  $N - 1$  do
5     if  $header[i]$  then
6        $slen[i] \leftarrow length[i]$ 
7     end
8     else
9        $slen[i] \leftarrow (slen[i - 1] - 1)$ 
10    end
11  end
12  assert( $header[0]$ )
13  for  $i = 1$  to  $N - 1$  do
14    assert( $header[i] \iff (slen[i - 1] - 1) = 0$ )
15  end
16  return ( $token, header$ )
17 end

```

算法 6 和算法 5 不立即等价，因为算法 6 从不可信见证值 $header[\cdot]$ 计算 $slen[\cdot]$ ，然后断言 $header[\cdot]$ 正确地从 $slen[\cdot]$ 计算，而算法 5 交错地计算 $slen[\cdot]$ 和 $header[\cdot]$ 。算法 6 的正确性可以通过对循环变量 i 的归纳来证明，本质上是在模拟算法 5 的控制流。若第 9 行的断言成立，则 $slen[0]$ 和 $header[0]$ 是正确的。若第 $(i - 1)$ 次的状态变量是正确的，且第 11 行的断言成立，则 $header[i]$ 是正确的， $slen[i]$ 也是正确的，因为 $slen[i]$ 的更新与两个算法相同。

在众多具体并行前缀算法中，我们使用 Sklansky [32] 形式（见算法 7），它使用 $O(n \log n)$ 个门和深度 $\lg n$ 。Brent-Kung 形式 [5] 等其他形式只需要 $O(n)$ 个门，但深度是 Sklansky 算法的两倍。对于求和校验所需的分层电路，值需要跨每一层传播，因此较低的深度更为可取，即使以额外的门为代价。

Algorithm 7: Sklansky [32] 并行前缀算法

```

1 procedure SCAN-ADD( $i_0, i_1, B[\ ]$ )
    输入: 原地操作, 设  $B_{\text{out}}[i_0] = B_{\text{in}}[i_0]$ , 对  $i_0 < i < i_1$  有
         $B_{\text{out}}[i] = B_{\text{in}}[i] + B_{\text{out}}[i - 1]$ 。
2     if  $i_1 - i_0 > 1$  then
3          $i_m \leftarrow i_0 + \lfloor (i_1 - i_0) / 2 \rfloor$ 
4         SCAN-ADD( $i_0, i_m, B$ )
5         SCAN-ADD( $i_m, i_1, B$ )
6         for 所有  $i: i_m \leq i < i_1$  do
7              $B[i] \leftarrow B[i] + B[i_m - 1]$ 
8         end
9     end
10 end

```

树结构重构 玩具 CBOR 文档最终表示为一棵树。可以将每个词元视为树节点: $\text{Int}(k)$ 词元代表一个叶节点, $\text{Array}(k)$ 词元代表一个有 k 个子节点的内部节点。在这棵树上, 我们将节点的树地址定义如下: 根节点的深度 $l = 0$, 树地址为数组 $[1]$ 。深度为 $(l + 1)$ 的节点的树地址为 $[p[0], p[1], \dots, (p[l] - 1), j]$, 其中 p 是父节点的树地址, j 是子节点编号。我们使用一些便于在电路中计算树地址的惯例。首先, 注意树地址中最后一个元素使用 $(p[l] - 1)$ 而非更自然的 $p[l]$ 。其次, 我们从右到左对子节点编号 (最后解析的节点的 $j = 1$, 左兄弟节点的子节点编号是其右兄弟节点的子节点编号加 1)。在这些惯例下, 树地址中的最后一个元素始终非零。

虽然树地址是可变长度的数组, 但在电路中我们将其表示为固定大小 L 的数组, 其中 L 是电路支持的最大嵌套深度, 隐式地用零填充。因此, 在实际电路中, 所有树地址存储在一个二维数组 $\text{counters}[i][l]$ 中, 按位置和嵌套深度索引。

假设我们拥有每个开始词元的位置 i (即 $\text{header}[i]$ 为真处) 的树地址, 我们就可以进行如下断言: ”深度 $(l + 1)$ 处的这个节点是深度 l' 处那个节点的子节点”, 方法是比较树地址的前缀 (直到 l)。因此, 拥有树地址允许我们生成检查文档句法结构断言的电路。

唯一剩余的问题是设计一个计算树地址的电路。我们的处理方式与分词器的处理方式相同, 首先给出计算树地址的序列算法, 然后将其转换为电路。我们的起点是算法 8, 它计算给定 $\text{counters}[i]$ 和分词器输出 $(\text{token}, \text{header})$ 的 $\text{counters}[i + 1]$ 。

Algorithm 8: 更新树地址的序列算法

```

1 procedure UPDATE-COUNTERS( $i, counters[][], token[], header[]$ )
    输入: 给定树地址  $counters[i]$  和分词器的输出  $(token, header)$ , 计算
         $counters[i + 1]$ 。
2    $counters[i + 1] \leftarrow counters[i]$ 
3   if  $header[i]$  then
4       设  $l[i] \in \{l \in \mathbb{N} : counters[i][l] \neq 0 \wedge \forall m > l : counters[i][m] = 0\}$ 
5        $counters[i + 1][l[i]] \leftarrow counters[i][l[i]] - 1$ 
6       if  $token[i] = \text{Array}(k)$  then
7            $counters[i + 1][l[i] + 1] \leftarrow k$ 
8       end
9   end
10 end

```

如果 $header[i]$ 为假, 则过程设 $counters[i] \leftarrow counters[i - 1]$, 如我们所假设的。否则, 设 l (在代码中显式称为 $l[i]$) 为满足 $counters[i][l] \neq 0$ 的最高深度, 即按树地址定义的树节点深度。因此, $counters[i + 1][l]$ 获得 $counters[i][l]$ 的左兄弟减一的树地址, 按我们关于子节点编号的惯例。若从位置 i 开始的词元是长度为 k 的数组, 则按惯例, 数组第一个 (最左) 子节点的子节点编号为 k , 因此树地址扩展一个深度, $counters[i + 1][l + 1] \leftarrow k$ 。

与分词器的情形相同, 原则上可以对所有 i 展开电路, 得到一个计算所有从 $counters[0] = [1, 0, 0, \dots]$ 开始的 $counters[i]$ 的深度较大的电路。为减少深度, 我们将 $l[i]$ 作为见证值提供, 以便在此时将 $counters[i][l]$ 的计算归约为 L 个分段扫描操作 (每个深度 l 一个), 加上见证值正确性的断言。

电路大小 表 3 展示了解析不同大小 CBOR 文档的代价, 这些文档大小对应于使用 n 个 SHA256 块哈希的消息大小。这些大小对应于完整 CBOR 语言的解析器, 而非我们用于说明目的的玩具 CBOR。表中指的是设计空间中的一个点, 但其他权衡也是可能的。例如, 在我们的实现中, 电路的输入由每个输入字节 9 个域元素构成: 8 个域元素对应待哈希输入的每一位, 第 9 个域元素以打包格式编码 $header$ 和 $l[i]$ 见证值。可以根据应用需求对输入大小和电路大小进行权衡。

注: 表中数据对应 $\mathbb{F}_{P256}$ 域。

5 评估

我们用约 25,500 行 C++ 代码实现了我们的 ZK 协议, 其中约 8,600 行对应单元测试或基准测试。我们的实现仅使用单线程; 我们报告的挂钟时间因此反映了整体计算性能。

表 3: 在素域上解析不同大小 CBOR 消息的电路大小和深度。消息大小对应于可以用 SHA256 在整数块数内哈希的消息。

消息大小 (字节)	深度	四次型数	项数	输入数
247	28	115,602	226,557	2,227
503	30	253,340	545,409	4,531
1079	34	616,331	1,559,052	9,715
1591	34	895,257	2,702,352	14,323
2231	36	1,343,877	4,588,287	20,083
2551	36	1,526,867	5,651,883	22,963

在本节中，我们报告若干匿名凭证操作应用的微基准测试和端到端基准测试。我们在 x86_64 机器和 Pixel Pro 6 手机上，使用 C++ 的 `benchmark` 包采集这些性能数据。x86_64 平台是 Google Cloud `central1-c` 可用区中运行的 `c4-highcpu-8` 实例。该机器配备四颗 Intel Xeon PLATINUM 8581C CPU（CPU 系列 6，型号 207），以 2.30GHz 频率运行，搭载 16GB RAM。Pixel Pro 6 平台配备 Google Tensor 处理器，8 核 12GB RAM。8 个核心由两个 2.80 GHz Cortex-X1、两个 2.25 GHz Cortex-A76 和四个 1.80 GHz Cortex-A55 组成。我们的基准测试在 2.80 GHz 的 Cortex-X1 处理器上运行。我们使用 SHA256 实现抗碰撞函数和随机预言机，由此得到 128 位计算安全参数。对于 Ligerio 承诺，我们打开 128 列，对应约 86 位的统计安全性。这些选择使我们的主应用的证明大小适配于 Android 框架的 *Intent* 缓冲区进程间通信。在手机上采集时序时，基准测试在性能核上运行，并限制为 1 秒运行，以减少热限流引起的测量误差。

我们以 Montgomery 形式存储素域元素，并在 64x64 位标量乘法器之上实现域乘法（即不使用 SIMD 指令）。对于 $\text{GF}(2^{128})$ ，我们在 arm64 和 x86_64 上均使用 128 位 SIMD 寄存器，并在 64x64 位无进位乘法器之上实现乘法，该乘法器在 x86_64 上由 `pclmul` 指令实现，在 arm64 AES 指令集上实现。我们目前不使用 AVX256、AVX512 或 GFNI。

5.1 代数原语

我们首先介绍代数原语的低级基准测试。表 4 中的第一个基准测试显示了我们的 NTT 实现在不同域和处理器上的性能。 $\mathbb{F}_{p_{128}}$ 和 $\mathbb{F}_{p_{64^2}}$ 域都至少具有 2^{32} 个单位根。如前所述， $\mathbb{F}_{p_{256}}$ 不具有单位根，因此我们必须使用二次扩展来引入单位根进行 NTT 计算。因此，在这个域上我们看到大约 4–5 倍的性能损失，相比 128 位素域。

Reed-Solomon 编码 表 5 报告了 Reed-Solomon 编码基准测试，因为这一步骤是 Ligerio 承诺的瓶颈。本节中的每个测试度量将大小为 n 的数组编码为大小 $4n$ 的码字的时间（即码率为 $1/4$ ）。回忆第 3 节，对于素域，我们的 Reed-Solomon 编码或多项式插值算法依赖于卷积，而卷积本身需要 2 次 NTT 和预计算常数的乘法。相反， $\text{GF}(2^{128})$ 域的编码使用 1 次双向 FFT 和与输入相同大小的 4 次 FFT。这里一个有趣的测量结果是，在单精度二次域 $\mathbb{F}_{p64^2}$ 上的编码相比 $\mathbb{F}_{p64}$ 有约 $3\times$ 的额外开销。

表 4: 不同域的 NTT 基准测试。x86 处理器比 Pixel 移动处理器快 1–2 倍。在所有情形下， $\text{GF}(2^{128})$ 域性能最优，至少快 $2\times$ 。 $\mathbb{F}_{p64}$ 虽非健全，但用于展示二次域扩展的开销。

		时间（毫秒）					
		2^{12}	2^{14}	2^{16}	2^{18}	2^{20}	2^{22}
x86_64	$\text{GF}(2^{128})$	0.042	0.195	0.883	5.17	26.7	134
	$\mathbb{F}_{p64}$	0.058	0.301	1.55	7.29	31.2	131
	$\mathbb{F}_{p64^2}$	0.175	0.860	3.89	17.2	77.0	353
	$\mathbb{F}_{p128}$	0.154	0.746	3.61	16.1	70.8	320
	$\mathbb{F}_{p256}$	1.41	6.70	32.7	146	660	2970
Pixel 6 Pro	$\text{GF}(2^{128})$	0.095	0.480	2.29	9.98	49.6	224
	$\mathbb{F}_{p64}$	0.086	0.404	2.12	9.19	42.8	201
	$\mathbb{F}_{p64^2}$	0.248	1.19	5.91	28.2	130	566
	$\mathbb{F}_{p128}$	0.231	1.12	5.53	26.8	123	538
	$\mathbb{F}_{p256}$	1.84	8.82	43.9	197	882	3870

与线性时间码的比较 Golovnev 等人 [19] 和 Xie、Zhang 和 Song [41] 提出使用线性时间可编码纠错码来构建多项式承诺方案。如何扩展这两种方案以支持线性和二次约束的 ZK 验证并不立即清晰（实际上，这两种方案都是为了解决基于求和校验的可验证计算协议中的最终查询而设计的）。此外，这类码的距离性质较差，因此需要对预言机进行大量查询，导致证明大小比我们的方案大一个数量级，实际上不实用。例如，Brakedown 方案的距离为 $1/20$ 、码率为 $3/5$ ，因此至少需要 6500 次查询。

我们还证明，对于相关参数范围，已知线性时间码比 Reed-Solomon 编码的实现慢。在表 6 中，我们对 128 位域上线性时间可编码码的 Brakedown 实现进行基准测试。由于该方案需要至少 6500 次对码字的查询，其基准测试从 2^{14} 开始。对于我们的参数范围，其线性时间编码的系数似乎大于我们在 $\text{GF}(2^{128})$ 中的准线性 Reed-Solomon 编码，我们的实现开始时快 $+5\times$ ，在 $n = 2^{20}$ 时仍快 $3\times$ 。

表 5: 不同域的 Reed-Solomon 编码性能。

		时间 (毫秒)					
		2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
x86_64	$\text{GF}(2^{128})$	0.037	0.175	0.794	3.75	21.5	113
	$\mathbb{F}_{p64}$	0.125	0.647	3.24	15.2	65.4	278
	$\mathbb{F}_{p64^2}$	0.366	1.83	8.40	36.3	166	757
	$\mathbb{F}_{p128}$	0.395	1.86	8.81	38.8	170	758
	$\mathbb{F}_{P256}$	3.15	14.7	71.2	316	1420	6360
Pixel 6 Pro	$\text{GF}(2^{128})$	0.085	0.408	2.10	9.81	45.5	219
	$\mathbb{F}_{p64}$	0.194	0.983	5.05	22.2	101.4	453
	$\mathbb{F}_{p64^2}$	0.560	2.79	13.6	63.6	289	1250
	$\mathbb{F}_{p128}$	0.582	2.94	14.3	66.7	298	1280
	$\mathbb{F}_{P256}$	4.16	19.6	97.2	432	1910	8510

表 6: 128 位域上线性时间可编码码的编码时间。这些码在 IOP 中至少需要 6500 次查询, 因此我们从 $n = 2^{14}$ 开始基准测试。这些码的速度略慢于我们的 RS $\mathbb{F}_{p64}$ 代码, 且比我们的 $\text{GF}(2^{128})$ 代码慢得多。

		时间 (毫秒)						所需查询数
		2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}	
Brakedown	x86_64	—	—	5.00	19.7	77.7	360	6593
	Pixel 6 Pro	—	—	6.09	25.3	98.6	466	6593

5.2 SHA256 哈希验证

许多 ZK 证明系统发布了关于证明某个值 e 的 SHA256 原像知识的基准测试。我们设计了一个以块操作数参数化的电路。我们的 $\text{SHA256}_n(e, m)$ 电路验证私密消息 m 在至多 n 个 SHA256 块操作 (即至多 $n \times 64 - 9$ 字节) 内可以哈希为公共值 e 。该电路对应于第 4.2 节中描述的位打包 2 的情形。1–32 块的时序结果见表 7。表 8 报告了我们关于 1 块 SHA256 的证明所需的承诺参数。

5.2.1 与 Ligerio 的比较

使用 Ligerio 验证相同函数是我们系统的自然替代方案。如引言中所述, Ligerio 需要承诺内部见证值的表格, 并且对每个二次约束承诺 3 个元素。因此, 对于给定电路 C , Ligerio 承诺的大小为 $O(|C|)$, 在实践中比验证 C 的相应验证抄本大很多倍。在本节中, 我们针对 SHA256 电路提供具体证据。

表 7: 我们关于 n 块消息对 SHA256 函数的原像知识 ZK 证明的时序基准测试。使用的电路是在 $\text{GF}(2^{128})$ 上描述的第 4.2 节中的电路。求和校验行报告为电路创建验证抄本所需的时间。总 ZK 证明者行报告总证明者时间（包括求和校验）以产生证明。

		我们的 ZK 协议，时间（毫秒）					
		$n = 1$	2	4	8	16	32
x86_64	验证抄本	6.07	12.1	24.3	49.8	106	228
	总 ZK 证明者	10.7	19.8	35.2	67.9	140	286
Pixel Pro 6	验证抄本	11.3	23.5	48.9	100.1	206	436
	总 ZK 证明者	19.2	37.0	68.9	132	257	517

表 8: 不同域中 1 块 SHA256-原像证明的承诺大小参数。

	$ w $	每行见证值数	编码行大小	总行数
$\mathbb{F}_p$	4,001	681	2^{13}	11
$\text{GF}(2^{128})$	4,228	681	2^{13}	12

一个问题是我们为协议生成的四次型电路被优化为减少四次型数量（即我们协议中的“四次型”），而 Ligerio 在总二次约束数量上表现更好的电路上性能更优。因此，为了进行公平比较，我们使用 `circom` 系统为上述 SHA256_n 电路族生成一个 R1CS 实例。回忆 R1CS 实例由见证值表格 w 和 $A \cdot B - C = 0$ 形式的约束列表构成，其中 A, B, C 是见证向量 w 的线性组合。较小的 R1CS 实例自然需要较少的乘法来验证。我们按照其论文中的描述将这个 R1CS 转换为 Ligerio 约束。具体地，对于每个 A, B, C ，我们引入新的 Ligerio 见证值 $\alpha_i, \beta_i, \kappa_i$ ，分别约束为等于见证 w 的线性组合。然后引入新的见证值 $\gamma_i = \alpha_i \cdot \beta_i$ ，并添加一个验证 $\gamma_i - \kappa_i = 0$ 的线性约束。总共，我们为每个 R1CS 约束引入 4 个新见证值、4 个线性约束和 1 个二次约束（许多 R1CS 约束具有空的 C 项，即 $C = 0$ ；在这种情形下，简单的优化会删除额外的见证值和第四个线性约束）。

综合来看，我们使用所有基础 R1CS 见证值加上 Ligerio 检查 R1CS 所需的额外见证值形成一个 Ligerio 承诺。表 9 报告了这两个数字。此外，每个见证值上的二次约束需要将两个项及其乘积的副本添加为见证值。所有这些见证值被划分为大小为 w 的行，附加约束条件是一个二次约束的三个见证值的副本必须在单独的行上且索引相同，每行只能包含一半数量。实现 ZK 还需要在每行中为随机填充保留位置，并添加 2 行额外的随机行。一系列额外约束控制 w 的大小。对于每个 Ligerio 承诺，我们遍历所有可行参数，并选择最小化承诺大小的参数集。这些最优选择也在表 9 中给出。注意，根据表 2，直接将我们的电路转换为 Ligerio ($n = 1$)

需要至少 167,348 个二次约束（比 R1CS 大约 3 倍）。

表 9: 由 n 块 SHA256 的最优 R1CS 表达式导出的 Ligerio 实例大小。为选择 Ligerio 使用的行大小，我们测试 w 的各种 2 的幂次以找到最小化证明大小的值。

		SHA256 _{n} ($\cdot$)		
		$n = 1$	2	4
R1CS	见证值数	29,853	60,381	121,437
	约束数	29,725	60,053	120,709
Ligerio	总见证值数	129,500	261,484	525,452
	二次约束数	29,725	60,053	120,709
	每行 w 的见证值数	3,149	6,426	6,426
	每行二次副本数	1,511	3,149	3,149
	编码行大小	2^{15}	2^{16}	2^{16}
	总行数	104	103	201

最后，我们对这些实例对我们的 Ligerio 实现进行基准测试，并在表 10 中报告时序。注意 Ligerio 最昂贵的组件是承诺阶段。为了健全性，Ligerio 必须使用 128 位域大小，这通常可以用较小域的 2 次或 4 次扩展来实现。我们尚未完全优化底层 NTT 以用于 Ligerio，因此我们使用 64 位素域来报告基准测试。虽然这一参数设置不如我们自己的实现健全，但它证明了我们的系统对于 $n = 1, 2, 4$ 块消息至少粗略快 20 倍，从而支持我们引言中的声明。可以确定的是，我们在表 10 中的数字大致与 Wang、Hazay 和 Venkitasubramaniam 在优化 Ligetron [38]（图 6(a)）实现中给出的数字匹配。该系统显示 $n = 1, 2, 4$ 块分别约需 250ms、450ms、750ms；然而他们的基准测试使用 4 线程在 4 核机器上，使用 50 位素域和更高的统计安全参数，因此比较并不精确。

我们系统与纯 Ligerio 约 $20\times$ 的性能提升可以通过检查见证值大小来部分解释。如表 8 所示，对于单个 SHA256 块，我们完整证明中的见证值包含约 9 倍更少的行（11 行对比 104 行），且块大小是 4 倍更小的，大致反映出产生承诺所需工作量减少了 36 倍；然而，我们的结果使用 128 位域，因此相对于表 10 中使用的 $\mathbb{F}_{p64}$ 有约 2 倍的额外开销。

5.2.2 与 Binius 的比较

Diamond 和 Posen [16, 17] 在 Binius [4] 系统中实现了承诺方案和可验证计算协议（即该协议不是零知识的）。他们提供了一个 SHA256 基准测试，验证 SHA 压缩函数的 32 次独立应用；他们的 x86_64 实现还利用了我们的系统未使用的 `gfni` 本机指令。在 x86_64 上，他们的基准测试在单核上运行 498ms（我们的 `verify`

表 10: 我们实现对验证 n 块消息 SHA256 原像的 Ligerio ZK 证明者时间。第一行报告承诺到见证值表格的时间。基准测试使用 $\mathbb{F}_{p64}$ 域运行，这对健全性不足，但代表 Ligerio 性能的上界。在 x86_64 和 Pixel 平台上，Ligerio 由于所需的较大承诺而本质上慢了约 20 倍。

		时间（毫秒）		
		$n = 1$	2	4
x86_64	承诺	222	384	736
	总 ZK 证明者	273	493	939
	相对本文开销	26×	25×	27×
	[38]	250	450	750
Pixel 6 Pro	承诺	294	536	1022
	总 ZK 证明者	380	717	1370
	相对本文开销	20×	19×	20×

在 228ms 内运行)，在 Pixel 上运行 904ms（我们的 436ms）。尽管他们的基准测试数字对于类似问题来说比我们的大，但随着 n 增大，性能差距会缩小；他们的系统融合了一些思路，也可能适用于我们的实现并在未来加速它。

5.2.3 与其他 ZK 系统的比较

STARK 系统是一种不需要可信设置参数且比其他证明系统（包括 Ligerio）产生更小证明的 ZK 论证系统。然而，如许多已发表的基准测试 [19] 所确认的，STARK 证明者时间大于 Ligerio 证明者时间。在未来工作中，我们计划对 Stark 的 SHA256 进行基准测试。

我们不对需要可信参数设置的证明系统进行基准测试，因为它们不满足我们的部署要求。尽管如此，先前工作的结果表明，这些系统倾向于具有需要更多工作的证明者算法。除了对电路大小的标准 NTT 之外，它们还需要在椭圆曲线群上进行多标量乘法例程，这通常代价较高。

5.3 ECDSA 签名验证

考虑 ZK 实例 (e, pk) 和对应协议，该协议验证存在 (r, s) 使得 $\text{ecdsaVerify}(pk, e, r, s) = 1$ 。如第 4.1 节所述，我们构造了一个需要 1085 个见证值以低深度完成验证的电路。表 11 报告了该问题的基本 ZK 基准测试。该基准测试证明，与流行观念相反，ECDSA 的零知识签名持有证明——大约需要 60ms——并不是构建凭证系统的实际瓶颈，并且与我们在下面展示的基本 BBS+ 签名持有证明相差一个小的倍数。

表 11: ECDSA 签名验证的时序基准测试。此处使用的电路在第 4.1 节中描述，基于 $\mathbb{F}_{p_{256}}$ 域。总 ZK 证明者时间由 Ligerio 承诺时间、生成验证抄本的时间和其他小步骤组成。

		时间（毫秒）		
		$n = 1$	2	3
x86_64	Ligerio 承诺	38.7	51.0	60.8
	生成验证抄本	13.5	26.5	38.6
	总 ZK 证明者	58.8	87.0	110
	总验证者	6.09	11.0	14.5
Pixel 6 Pro	Ligerio 承诺	51.0	67.2	80.0
	生成验证抄本	20.3	40.1	58.4
	总 ZK 证明者	80.5	120.0	152.0
	总验证者	8.50	16.2	21.2

与 BBS+ 方案的比较 为将我们的结果与 BBS+ 进行比较，BBS+ 在文献中提供了最快的签名持有知识零知识证明，我们对 `pairing_crypto` 包 [30] 进行基准测试，配置为使用 BLS12-381 曲线和 SHA256，签名包含 100 个属性，展示时披露 3 个属性。`criterion` 包给出了基准测试结果。在 x86_64 上，BBS+ 证明生成步骤需要 10.2ms，验证步骤需要 8ms；我们的 ECDSA 证明生成方法在约 6× 的性能范围内。然而，BBS+ 微基准测试没有考虑执行凭证展示操作所需的额外任务，例如验证证书未过期。

6 应用

在本节中，我们将我们的匿名凭证系统应用于数字身份领域。

6.1 简单匿名凭证

为说明我们系统的组件如何在匿名凭证系统中使用，我们设计了一种基于护照机读区（MRZ）格式的玩具文档格式用于凭证。

在图 2 中，我们描述了一种 183 字节的凭证，它允许编码至少 60 个标准属性，以及用于设备绑定检查的设备公钥和指示有效期的时间值。值得注意的是，这种格式仅需 3 个 SHA256 块来哈希，并且无需解析即可确定属性的字节位置。对于这种凭证，我们开发了一个简单基准测试，用于证明凭证持有者年龄大于 18 岁（这相当于检查格式中第 74 字节的值）。具体的定理见算法 9。

firstname				
lastname				
dob	G	age over x	validFrom	validUntil
dpk_x				
dpk_y				
issuerid	Attributes			

图 2: 一个 183 字节凭证的简单文档格式。

Algorithm 9: 玩具凭证验证**要求:** $x = (\text{pk}, a, id, tr, \text{now})$, $w = (\text{MSO}, pk_{dx}, pk_{dy})$

- 1 验证以下约束:
- 2 $e_1 = \text{SHA256}(\text{MSO}[0 : 183])$
- 3 $a = \text{MSO}[id]$
- 4 $(pk_{dx}, pk_{dy}) = \text{MSO}[96 : 160]$
- 5 $t_{\text{start}} = \text{MSO}[48 : 56]$
- 6 $t_{\text{end}} = \text{MSO}[56 : 64]$
- 7 $t_{\text{start}} < t_{\text{now}} < t_{\text{end}}$
- 8 **true** = $p256.\text{verify}((r_1, s_1), e_1, PK_{II})$
- 9 **true** = $p256.\text{verify}((r_2, s_2), H(tr \| hdr), (pk_{dx}, pk_{dy}))$

表 12 中的基准测试表明, 这种玩具格式实现了与 BBS+ 凭证验证时间相差小倍数的性能 (未考虑格式化 BBS+ 属性或检查凭证有效性的时间)。

表 12: $\mathbb{F}_{p256}$ 上的玩具凭证基准测试。

	深度	四次型数	项数	输入数
玩具电路	12	276,465	818,533	10,123
证明者时间 (毫秒)		证明大小 (KB)		
x86		332		291
Pixel Pro 6		470		291

6.2 MDOC 匿名凭证

ISO 标准 18013-5 [1] 规定了一种广泛应用于移动驾驶证和国家身份格式的数字身份格式。例如, 亚利桑那州、加利福尼亚州、科罗拉多州、乔治亚州、马里兰州和新墨西哥州均以 MDOC 格式向某些移动设备颁发驾驶证。

MDOC 格式支持一种选择性披露，允许 MDOC 持有者披露凭证中所断言的属性子集。这种披露机制通过在颁发方签名的凭证格式中包含属性值的加盐哈希来实现。然而，加盐哈希技术存在两个众所周知的隐私缺陷：若 Alice 对依赖方 P_1 和 P_2 使用同一 MDOC 凭证来断言身份，则 P_1 和 P_2 可以结合其对展示协议的视角来跨会话追踪用户。例如，过期时间或设备密钥值可以用于跨不同依赖方追踪用户或将用户链接。一个简单的对策是要求用户对每个依赖方使用不同的 MDOC——这一措施对颁发方来说代价高昂，颁发方现在必须向每个用户颁发数千份 mdoc，对用户来说也代价高昂，用户必须维护状态以确保唯一性。一个更严重的缺陷是颁发方可以与依赖方协作，以便跨会话识别用户。这一缺陷延迟了 MDOC 在某些应用中的部署。

用于 MDOC 的 ZK 针对 MDOC 格式中颁发方-依赖方勾结问题的解决方案是修改展示协议，使用户转而产生一个“其 mdoc 针对所请求属性验证通过”的零知识证明。具体地，我们旨在让验证者相信算法 10 所示定理陈述的真实性，同时不泄露任何额外信息：

Algorithm 10: MDOC 验证

要求: $x = (PK_{II}, attr, Z, tr, time)$, w 由一个 2231 字节字符串 MSO、一个长度 len 、一个哈希 e_1 、一个哈希 h_2 、一个索引 X 、一对签名 (r_1, s_1) 、一个 32 字节随机数 $nonce$ 、一对 (pk_{dx}, pk_{dy}) 、一对字符串 t_{start}, t_{end} 、一个头部 hdr 和一个签名 (r_2, s_2) 构成

- 1 验证以下关系：
 - 2 $e_1 = \text{SHA256}(\text{MSO}[0 : len])$
 - 3 $h_2 = \text{MSO}[\text{valueDigests}][\text{org.iso.18013.5.1}][X]$
 - 4 $h_2 = \text{SHA256}(nonce, attr, Z)$
 - 5 $(pk_{dx}, pk_{dy}) = \text{MSO}[\text{deviceKeyInfo}][\text{deviceKey}][-2, -3]$
 - 6 $t_{start} = \text{MSO}[\text{validityInfo}][\text{validFrom}]$
 - 7 $t_{end} = \text{MSO}[\text{validityInfo}][\text{validUntil}]$
 - 8 $t_{start} < t_{now} < t_{end}$
 - 9 **true** = $p256.verify((r_1, s_1), e_1, PK_{II})$
 - 10 **true** = $p256.verify((r_2, s_2), H(tr || hdr), (pk_{dx}, pk_{dy}))$
-

重要的是，该定理的公共值是 $(PK_{II}, attr, Z, tr, time)$ ——这些值由证明者（持有 mdoc 的用户）和验证者（依赖方）共享。其中 PK_{II} 是身份颁发方的公钥， Z 是要披露的属性值（例如，布尔值 `age_over_18`）， $attr$ 是该属性的名称（作为区分不同属性的标识符）， tr 是活性抄本， t_{now} 用于验证 mdoc 未过期。陈述中其余值是隐藏的，即验证者相信这些值的存在，但不通过交互学习它们。

根据表 13，在 X86 处理器上，我们的基准测试证明者运行 1.15 秒，验证者运行 0.52 秒，在 Pixel 6 Pro 上分别为 1.17 秒和 0.68 秒。该基准测试支持最长 2231

字节的 mdoc，证明中最昂贵的部分涉及 MSO 的 SHA256 哈希。我们的基准测试实现了双域优化，SHA256 电路在 $GF(2^{128})$ 中验证。

表 13: 针对至多 2231 字节 MSO 的年龄大于 18 属性的 ZK MDOC 展示的证明者时间。电路对应算法 10。

	证明者（毫秒）	验证者（毫秒）
x86_64	1150	520
Pixel 6 Pro	1170	680

扩展 这一基本应用可以方便地扩展以支持更多隐私特性。例如，可以隐藏颁发方的身份，转而证明颁发方是某个经过认证的颁发方列表中的成员。还可以以几乎零额外代价支持同时披露多个属性。最后，可以使用标准 ZK 列表成员资格技术以零知识方式支持 MDOC 凭证的吊销。在未来工作中，我们计划纳入这些扩展。

致谢

John Kuszmaul 在我们组实习期间实现了第 3.1 节中讨论的 Reed-Solomon 编码算法。我们感谢 Sergeui Alleko 提供的代码审查与有益讨论。我们感谢 Quan Nguyen、Cory Barker、Justin Brickell、Bogdan Brinzarea Iamandi、Gareth Oliver 和 David Zeuthen 提出的有益意见与讨论。我们还感谢 Justin Thaler、Muthu Venkitasubramaniam、Carmit Hazay、Jonathan Katz 和 Mark Moir 提供的有益评论和相关工作参考。我们特别感谢 Shiv Venkataraman、Sunita Verma 和 Joshy Joseph 对本项目的资助与支持。

参考文献

- [1] ISO/IEC FDIS 18013-5. Personal identification—iso-compliant driving licence—part 5: Mobile driving licence (mdl) application, 2021.
- [2] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Liger: Lightweight sublinear arguments without a trusted setup. *eprint/2022/1608 and CCS'17*, 2022.
- [3] Carsten Bormann and Paul E. Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, December 2020.
- [4] Binius implementation. gitlab.com/IrreducibleOSS/binius.git.

- [5] Brent and Kung. A regular layout for parallel adders. *IEEE Transactions on Computers*, C-31(3):260–264, 1982.
- [6] Guy E. Blelloch. Prefix sums and their applications. Technical Report CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, November 1990.
- [7] Michael Ben-Or, Oded Goldreich, Shafi Goldwasser, Johan Håstad, Joe Kilian, Silvio Micali, and Phillip Rogaway. Everything provable is provable in zero-knowledge. In *CRYPTO’88*, 1988.
- [8] Stefan Brands. *Rethinking Public Key Infrastructure and Digital Certificates*. PhD thesis, Eindhoven Institute of Technology, 1999.
- [9] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-shamir from simpler assumptions. In *eprint/2018/1004*, 2018.
- [10] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous attestation using the strong Diffie Hellman assumption revisited. In *Cryptology ePrint Archive, Paper 2016/663*, 2016.
- [11] Certicom. Sec 1: Elliptic curve cryptography, v2.0. Standards document, 2009.
- [12] David Chaum. Security without identification: Transaction systems to make big brother obsolete. *Communications of the ACM*, 10(28):1030–1044, 1985.
- [13] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Eurocrypt’2001*, 2001.
- [14] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. Practical verified computation with streaming interactive proofs. In *ITCS*, pages 90–112, 2012.
- [15] Antoine Delignat-Lavaud, Cedric Fournet, Markulf Kohlweiss, and Bryan Parno. Cinderella: Turning shabby x.509 certificates into elegant anonymous credentials with the magic of verifiable computation. In *Oakland IEEE S&P 2016*, 2016.
- [16] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. *Cryptology ePrint Archive, Paper 2023/1784*, 2023.
- [17] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. *Cryptology ePrint Archive, Paper 2024/504*, 2024.

- [18] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: interactive proofs for muggles. In *STOC'08*, 2008.
- [19] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic snarks for r1cs. In *eprint 2021/1043*, 2021.
- [20] Roderick Gow. Cauchy's matrix, the Vandermonde matrix and polynomial interpolation. *Bulletin of the Irish Mathematical Society*, 28:45–52, March 1992.
- [21] Donald E. Knuth. *The art of computer programming, volume 2 (3rd ed.): seminumerical algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 1997.
- [22] Ahmed Kosba, Zhichao Zhao, Andrew Miller, Yi Qian, T-H. Hubert Chan, Charalampos Papamanthou, Rafael Pass, abhi shelat, and Elaine Shi. C0C0: A framework for building composable zero-knowledge proofs. *eprint/2015/1093*, 2015.
- [23] Sian-Jheng Lin, Tareq Y. Al-Naffouri, Yunghsiang S. Han, and Wei-Ho Chung. Novel polynomial basis with fast Fourier transform and its application to reed-solomon erasure codes. *IEEE Transactions on Information Theory*, 62(11):6284–6299, 2016.
- [24] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 316–325, 2014.
- [25] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast Fourier transform, 2018.
- [26] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *J. ACM*, 39:859–868, October 1992.
- [27] Anna Lysyanskaya, Ron Rivest, Amit Sahai, and Stefan Wolf. Pseudonym systems. In *Selected Areas in Cryptography*, 1999.
- [28] Todd Mateer. *Fast Fourier transform algorithms with applications*. PhD thesis, Clemson University, 2008.
- [29] Henri J. Nussbaumer. Fast polynomial transform algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28:205–215, 1980.

- [30] Pairing crypto bbs implementation. https://github.com/mattrglobal/pairing_crypto, 2024.
- [31] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. zk-creds: Flexible anonymous credentials from zksnarks and existing identity infrastructure. In *Oakland IEEE S&P*, 2023.
- [32] Jack Sklansky. Conditional-sum addition logic. *IRE Trans. Electron. Comput.*, 9:226–231, 1960.
- [33] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *CRYPTO’13*, 2013.
- [34] Justin Thaler. Proofs, arguments, and zero-knowledge. Manuscript, 2022.
- [35] Joris van der Hoeven. The truncated fourier transform and applications. In *Proceedings of the 2004 International Symposium on Symbolic and Algebraic Computation*, ISSAC ’04, page 290–296, New York, NY, USA, 2004. Association for Computing Machinery.
- [36] Joris van der Hoeven. Notes on the truncated Fourier transform, 2005.
- [37] Joachim von zur Gathen and Jürgen Gerhard. Arithmetic and factorization of polynomial over $\mathbb{F}_2$ (extended abstract). In *Proceedings of the 1996 International Symposium on Symbolic and Algebraic Computation*, ISSAC ’96, pages 1–9, New York, NY, USA, 1996. Association for Computing Machinery.
- [38] Ruihan Wang, Carmit Hazay, and Muthuramakrishnan Venkitasubramaniam. Ligetron: Lightweight scalable end-to-end zero-knowledge proofs post-quantum zk-snarks on a browser. In *Oakland IEEE S&P’24*, 2024.
- [39] Riad S Wahby, Ye Ji, Andrew J Blumberg, abhi shelat, Justin Thaler, Michael Walfish, and Thomas Wies. Full accounting for verifiable outsourcing. In *ACM CCS’2017*, 2017.
- [40] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *eprint/2017/1132 and Oakland S&P’2018*, 2018.
- [41] Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In *eprint/2022/1010*, 2022.

- [42] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. Transparent polynomial delegation and its applications to zero knowledge proof. Cryptology ePrint Archive, Paper 2019/1482, 2019.

A 卷积插值

设 $p(k)$ 为次数至多为 d 的多项式。给定 $p(i)$ 在整数点 $0 \leq i \leq d$ 处的赋值, 我们希望计算 $p(k)$ 在某个范围内所有整数 k 处的值。我们的下一个目标是将这个多项式插值表示为单一卷积运算。

引理 A.1 ((Lagrange 插值)). 设 $p(k)$ 为定义在某个域上的次数至多为 d 的多项式。对所有 k , 我们有

$$p(k) = \sum_{0 \leq i \leq d} \binom{k}{i} \binom{d-k}{d-i} p(i). \quad (5)$$

证明. 设 $q(k)$ 为方程 (5) 的右端。我们希望证明 $p = q$ 。

二项式系数 $\binom{x}{r}$ 是 x 的次数至多为 r 的多项式。因此 $q(k)$ 是次数至多为 d 的多项式。我们现在证明 $p(k) = q(k)$ 对 $0 \leq k \leq d$ 成立, 因此两个多项式相同。

假设 $0 \leq k \leq d$ 。若 $i = k$, 则第 i 项为 $p(k)$ 。若 $i > k$, 则 $\binom{k}{i} = 0$, 故第 i 项为 0。若 $i < k$, 则 $\binom{d-k}{d-i} = 0$, 故第 i 项也为 0。因此 $q(k) = p(k)$, 如所求。 $\square$

引理 A.2 ((卷积形式的 Lagrange 插值)). 设 $p(k)$ 为定义在某个域上的次数至多为 d 的多项式。对 $k > d$, 我们有

$$p(k) = (-1)^d \cdot (k-d) \binom{k}{d} \cdot \sum_{0 \leq i \leq d} \left(\frac{1}{k-i} \right) \cdot (-1)^i \binom{d}{i} p(i). \quad (2)$$

证明. 证明是对方程 (5) 的直接操作, 将依赖 k 的项与依赖 i 和 $k-i$ 的项分离。

我们使用熟知的二项式指标取反恒等式

$$\binom{-r}{s} = (-1)^s \binom{r+s-1}{s}$$

使 $\binom{d-k}{d-i}$ 中的上指标非负, 如下:

$$\binom{k}{i} \binom{d-k}{d-i} = (-1)^{d-i} \binom{k}{i} \binom{k-i-1}{d-i}.$$

将二项式系数展开为阶乘并化简, 我们得到:

$$\begin{aligned}
 (-1)^{d-i} \binom{k}{i} \binom{k-i-1}{d-i} &= (-1)^{d-i} \frac{k!}{i!(k-i)!} \cdot \frac{(k-i-1)!}{(d-i)!(k-d-1)!} \\
 &= \frac{(-1)^{d-i}}{k-i} \cdot \frac{k!}{i!(d-i)!(k-d-1)!} \\
 &= \frac{(-1)^{d-i}}{k-i} \cdot \frac{1}{d!} \cdot \binom{d}{i} \cdot (k-d) \cdot d! \cdot \binom{k}{d} \\
 &= \frac{(-1)^{d-i}}{k-i} \cdot \binom{d}{i} \cdot (k-d) \cdot \binom{k}{d} \\
 &= (-1)^d \cdot \frac{1}{k-i} \cdot (-1)^i \binom{d}{i} \cdot (k-d) \binom{k}{d},
 \end{aligned}$$

由此得到公式 (2) (即方程 (2))。 □

注记. 等式

$$(k-d) \binom{k}{d} = k \binom{k-1}{d}$$

提供了一种在 k 取某个区间内的值时线性时间计算 $\binom{k}{d}$ 的方法。

B MDOC 标准

MDOC 在 ISO 18013-5 中规定。这里我们总结其中与一个二进制字符串相关的相关部分, 该字符串可以被解析为如下形式的层次化 MSO[] 对象:

```

{"version": "1.0",
 "digestAlgorithm": "SHA-256",
 "docType": "org.iso.18013.5.1.mDL",
 "valueDigests": {
   "org.iso.18013.5.1":
     {13: h'B628...69D2',
      11: h'6F94...5BD4',
      ...
14: h'8CFE...604C',
  ...
   "org.iso.18013.5.1.aamva": {
     15: h'1034...402A',
     ...
     8: h'7AC6...8485'}
  },
 "deviceKeyInfo": {

```

```

        "deviceKey": {1: 2, -1: 1,
                      -2: h'7B8F...F321',
                      -3: h'859E...772C'}
    },
    "validityInfo": {
        "signed": 0("2023-10-11T13:18:15Z"),
        "validFrom": 0("2023-10-11T13:18:15Z"),
        "validUntil": 0("2023-11-10T13:18:15Z")
    }
}

```

在我们的上下文中，一个 mdoc 凭证由这个 MSO 对象的二进制序列化构成，该对象由身份颁发方 (II) 签名，其公开签名密钥 PK_I 为众所周知。我们将签名记为 (r_1, s_1) 。

考虑字符串 `MSO[valueDigests][org.iso.18013.5.1]` [14]，它表示由缩写为 $(id, nonce, attrid, attrvalue)$ 的字符串元组构成的哈希：

```

"digestID": 14,
"random": h'8c082af3d5d78b98bcc00bd26fae5130',
"elementIdentifier": "age_over_18",
"elementValue": true

```

用户可以在 mdoc 凭证中选择性地披露 `age_over_18` 属性 id，方法是提供 MSO 的签名，以及一个完整原像（由那 4 个字符串构成），对应于 MSO 中包含的 ... [14] 哈希。这本质上断言了属性“`age_over_18`”被身份颁发方签署的捆绑中包含，同时隐藏了其他已签署的属性。

是什么阻止用户通过复制凭证来破坏健全性？注意 `MSO[deviceKeyInfo][deviceKey][-2]` 和 `[-3]` 表示一个公开签名密钥 PK_d ，它对应于保存在用户设备安全元件中的私钥。

当用户尝试展示 mdoc 时，除了 (r_1, s_1) 和属性的原像之外，用户的设备还在身份请求抄本上用 PK_d 签名密钥计算签名 (r_2, s_2) 。假设颁发 mdoc 时，身份颁发方验证了 SK_d 被安全地存储在设备中，则这第二个签名建立了设备绑定健全性。